

Práctica Guiada 1: IDEs (Parte 1)

[Actividad 1. Instalación del IDE \(Visual Studio Code\)](#)

[Actividad 2. Ejecución del primer script](#)

[¿En qué lenguaje de programación está escrito el script_1?](#)

[¿Has podido ejecutarlo directamente nada más instalar Visual Studio Code?](#)

[¿Has tenido que instalar algún plugin adicional? ¿Cuál?](#)

[Actividad 3. Lenguaje de programación y plugins](#)

[Lenguaje de programación de los scripts](#)

[Plugins instalados:](#)

[Actividad 4. Depuración de código \(modo Debug\) y puntos de ruptura](#)

[Introducción](#)

[Actividad 4.1. Depuración del script 1.py](#)

[Modos de ejecución probados:](#)

[Diferencias observadas:](#)

[Actividad 4.2. Depuración del script_2.py:](#)

[Actividad 4.4 – Ejecución del script_4.py:](#)

[Actividad 4.5 – Depuración del script 5.py:](#)

[Actividad 4.6:](#)

[Actividad 4.7 – Ejecución del script_7.py:](#)

[Actividad 5. Configuración del IDE para otro lenguaje: Java:](#)

[Actividad 6. Compilación y generación del Bytecode \(Java\):](#)

[Actividad 6 – Uso del IDE IntelliJ IDEA:](#)

[Actividad 7:](#)

[Actividad 7 \(Depurar3\):](#)

[Actividad 8:](#)

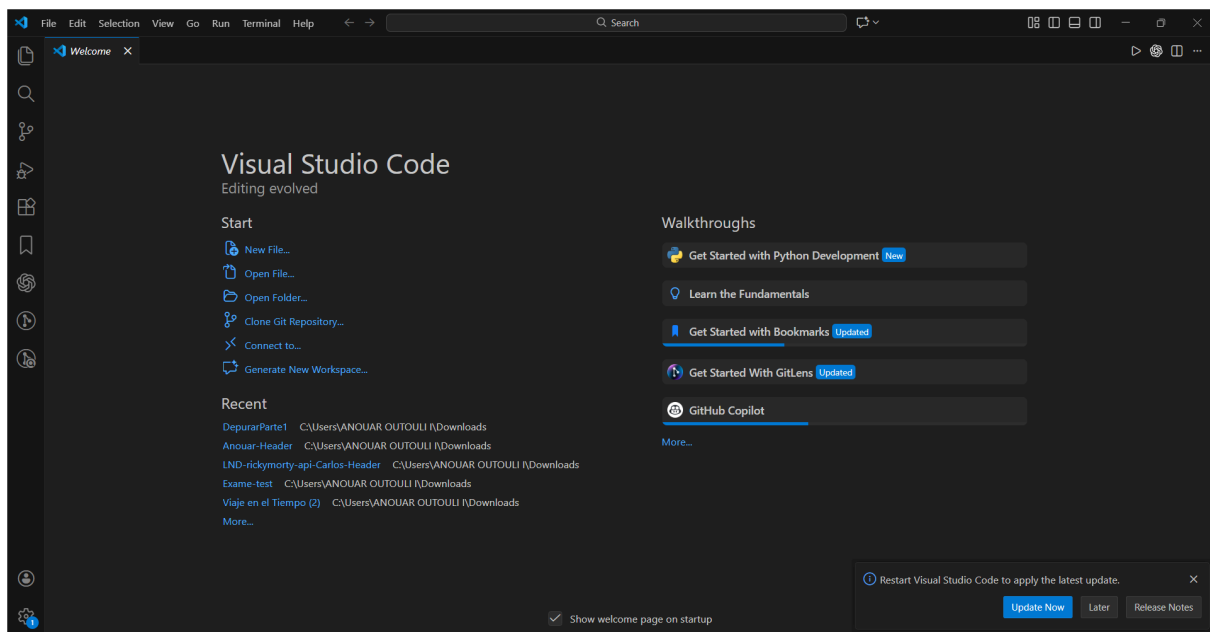
[Actividad 9:](#)

Actividad 1. Instalación del IDE (Visual Studio Code)

Para la realización de esta práctica se ha instalado el IDE **Visual Studio Code** en el sistema operativo correspondiente.

La descarga se ha realizado desde la página oficial de Visual Studio Code, seleccionando la versión adecuada al sistema operativo.

El proceso de instalación se ha completado correctamente y el IDE se ha ejecutado sin errores, quedando listo para su uso en las siguientes actividades prácticas.



Actividad 2. Ejecución del primer script

¿En qué lenguaje de programación está escrito el script_1?

El archivo `script_1` está escrito en el lenguaje de programación **Python**.

¿Es un lenguaje compilado o interpretado?

Python es un lenguaje **interpretado**, lo que significa que el código se ejecuta directamente mediante un intérprete, sin necesidad de un proceso previo de compilación.

¿Has podido ejecutarlo directamente nada más instalar Visual Studio Code?

No, no ha sido posible ejecutar el script únicamente con la instalación de Visual Studio Code, ya que el IDE no incluye por defecto el intérprete de Python.

¿Qué tienes que instalar para ejecutarlo?

Para poder ejecutar el script ha sido necesario instalar:

- El intérprete de Python
- La extensión de Python para Visual Studio Code

¿Has tenido que instalar algún plugin adicional? ¿Cuál?

Sí, se ha instalado la extensión **Python (Microsoft)** para Visual Studio Code.

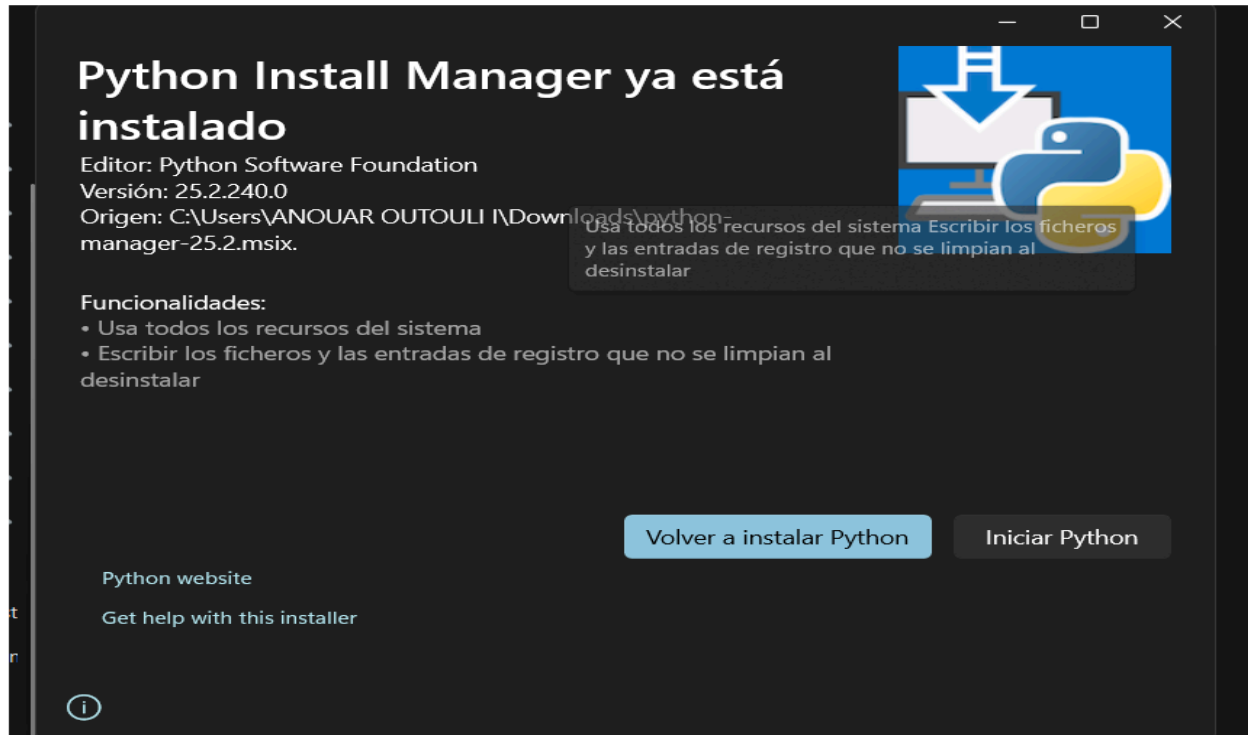
Esta extensión permite ejecutar código Python, depurarlo y facilita la escritura del código mediante autocompletado y detección de errores.

Ejecución del script

Una vez instalado Python y la extensión correspondiente, se ha abierto el archivo `script_1.py` desde Visual Studio Code y se ha ejecutado correctamente, mostrando el resultado esperado por la terminal.

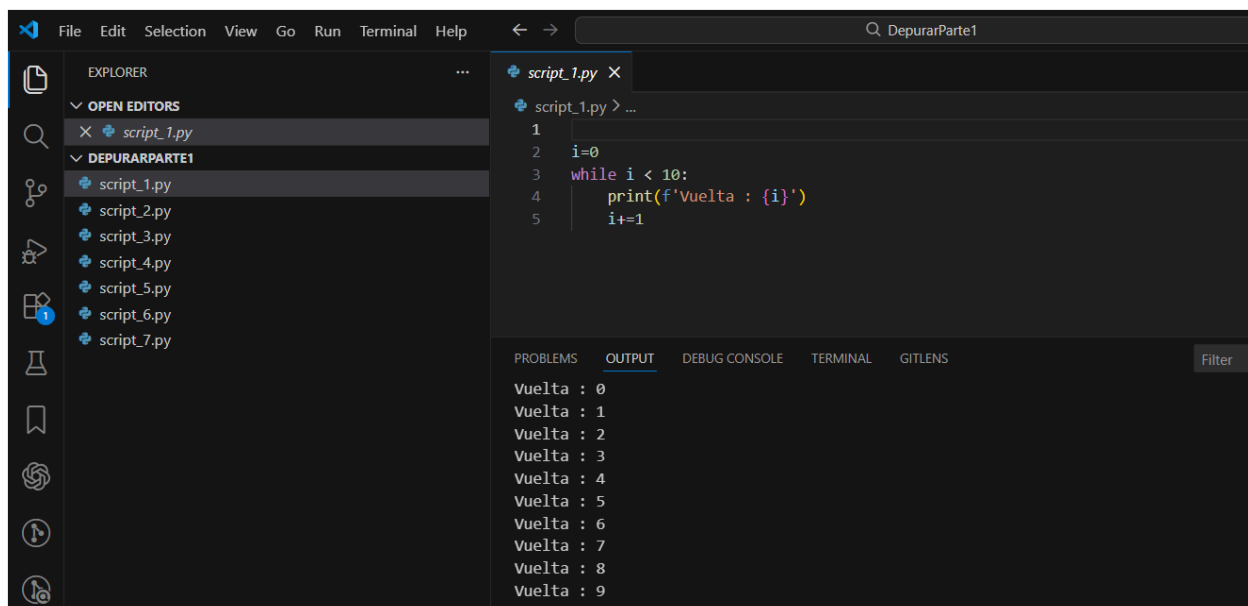
Captura 5: Extensión de Python instalada en Visual Studio Code





Captura 4: Carpeta del proyecto abierta en Visual Studio Code con el archivo `script_1.py`.

Ejecución correcta del script mostrando la salida por pantalla.



Actividad 3. Lenguaje de programación y plugins

Lenguaje de programación de los scripts

Los scripts utilizados en esta práctica están escritos en el lenguaje de programación **Python**.

Python es un lenguaje interpretado, de alto nivel y multiplataforma, ampliamente utilizado en el ámbito de las tecnologías de la información.

Principales características de Python

- Lenguaje interpretado
- Sintaxis clara y fácil de leer
- Multiplataforma
- Gran cantidad de librerías y módulos
- Permite distintos paradigmas de programación

Puntos fuertes de Python

- Facilidad de aprendizaje
- Desarrollo rápido de aplicaciones
- Gran comunidad y documentación
- Muy utilizado actualmente en el sector tecnológico

Puntos débiles de Python

- Menor rendimiento que los lenguajes compilados
- Dependencia del intérprete para su ejecución
- No es el más adecuado para aplicaciones de muy bajo nivel

Principales campos de aplicación de Python

- Desarrollo web
- Automatización de tareas
- Ciencia de datos
- Inteligencia artificial
- Scripting y testing

Plugins instalados en Visual Studio Code

Para trabajar de forma más eficiente con Python en Visual Studio Code se han instalado las siguientes extensiones oficiales de Microsoft:

Plugins instalados:

- **Python**
Proporciona soporte completo para el lenguaje Python, permitiendo ejecutar scripts, depurar código y mejorar la productividad en el IDE.
- **Pylance**
Servidor de lenguaje que mejora el análisis del código, el autocompletado y la detección de errores en tiempo real.
- **Python Environments**
Permite gestionar y seleccionar de forma sencilla los entornos de Python desde Visual Studio Code.



Actividad 4. Depuración de código (modo Debug) y puntos de ruptura

Introducción

El modo **Debug** permite ejecutar el código paso a paso, facilitando la detección de errores y la observación del comportamiento del programa durante su ejecución.

Para ello se utilizan los **puntos de ruptura (breakpoints)**, que detienen la ejecución del programa en una línea concreta del código.

Actividad 4.1. Depuración del script_1.py

Se ha colocado un punto de ruptura en el archivo `script_1.py` y se ha ejecutado el programa en modo Debug desde Visual Studio Code.

Durante la ejecución paso a paso se han observado los valores de las variables y la salida que se va mostrando por pantalla.

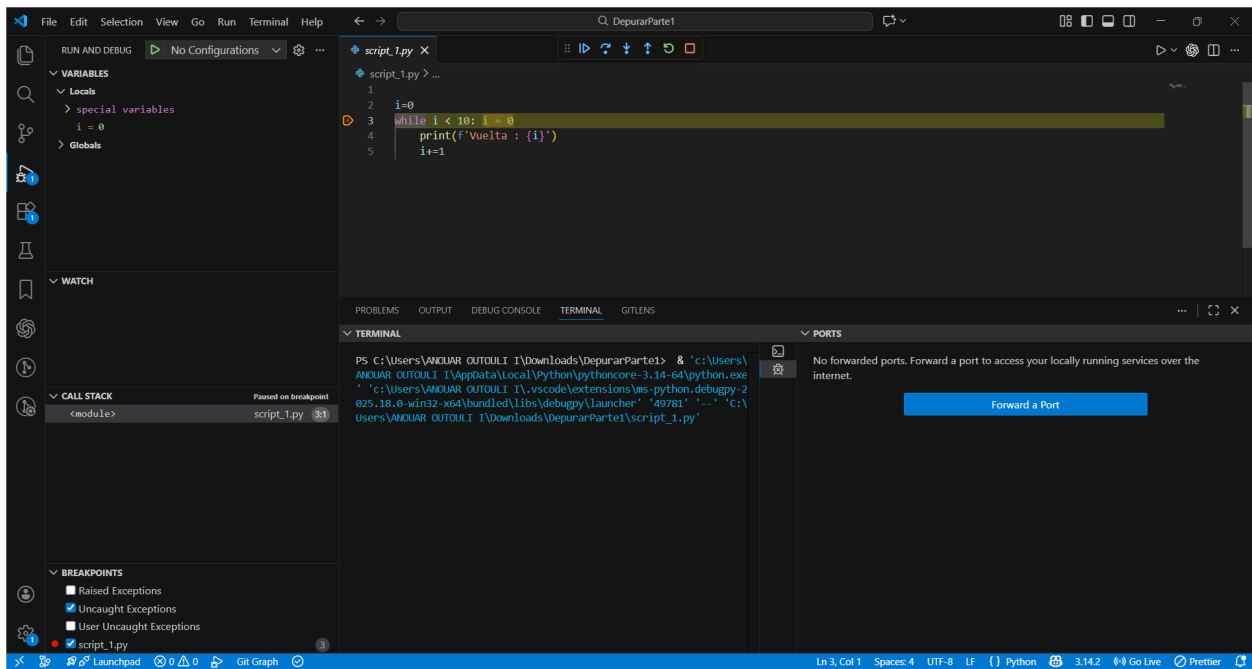
Modos de ejecución probados:

- **Continue:** Ejecuta el programa de forma normal hasta el siguiente punto de ruptura.
- **Step Over:** Ejecuta el código línea a línea sin entrar en las funciones.
- **Step Into:** Ejecuta el código línea a línea entrando en las funciones.
- **Step Out:** Sale de la función actual y continúa la ejecución.

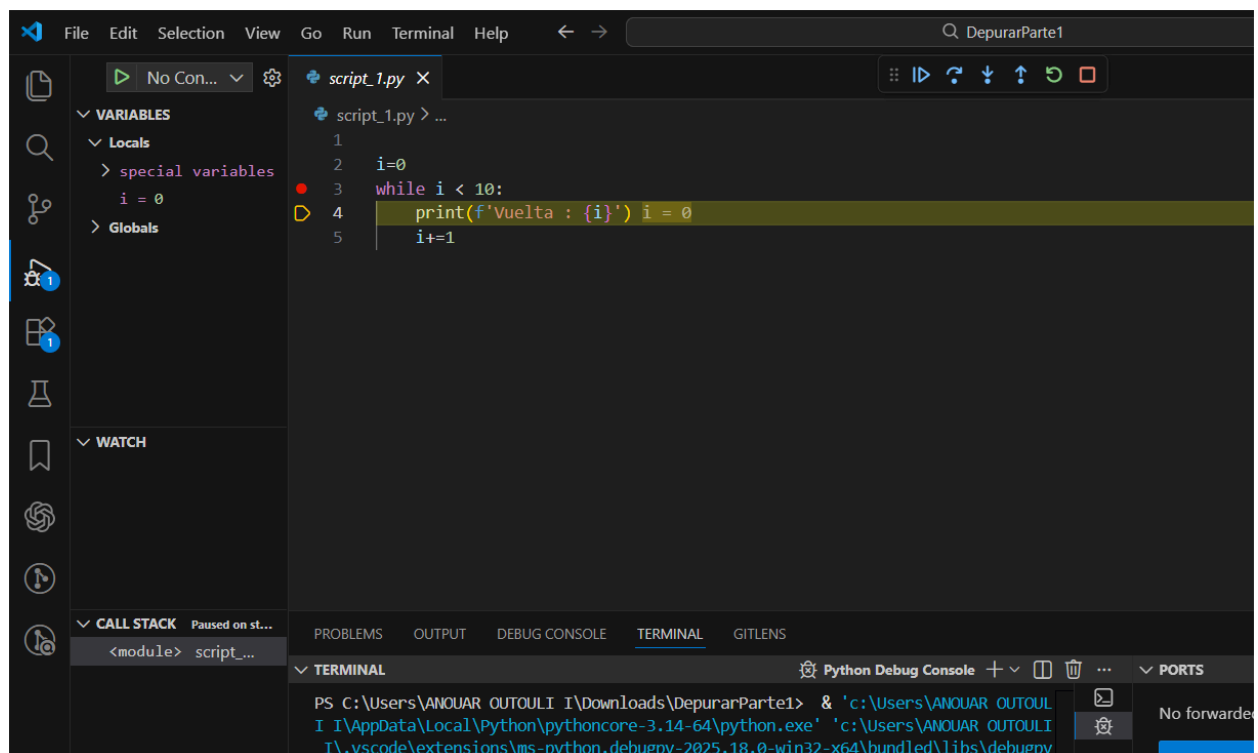
Diferencias observadas:

En este script, las diferencias entre los modos de ejecución son poco apreciables, ya que el código es sencillo y no hace un uso intensivo de funciones.

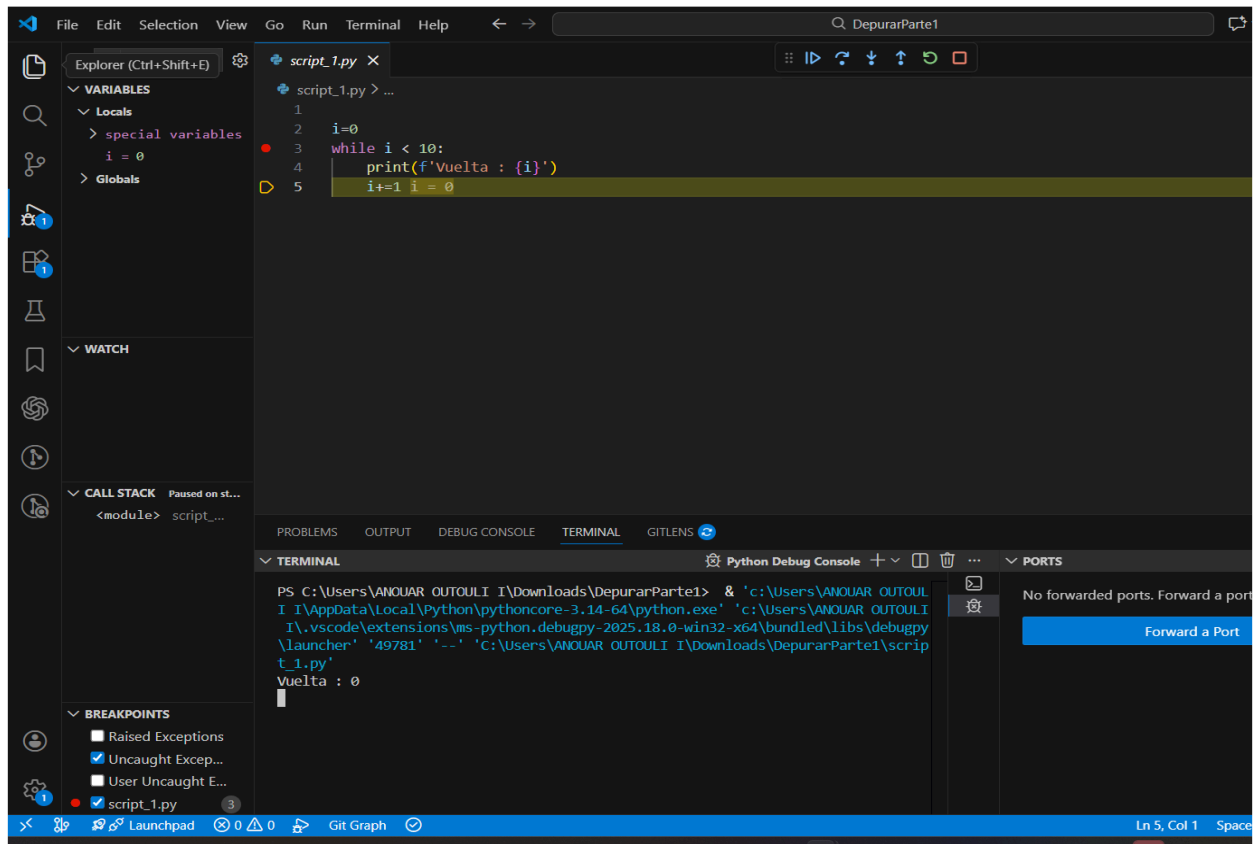
Captura 1: Punto de ruptura colocado en `script_1.py`.



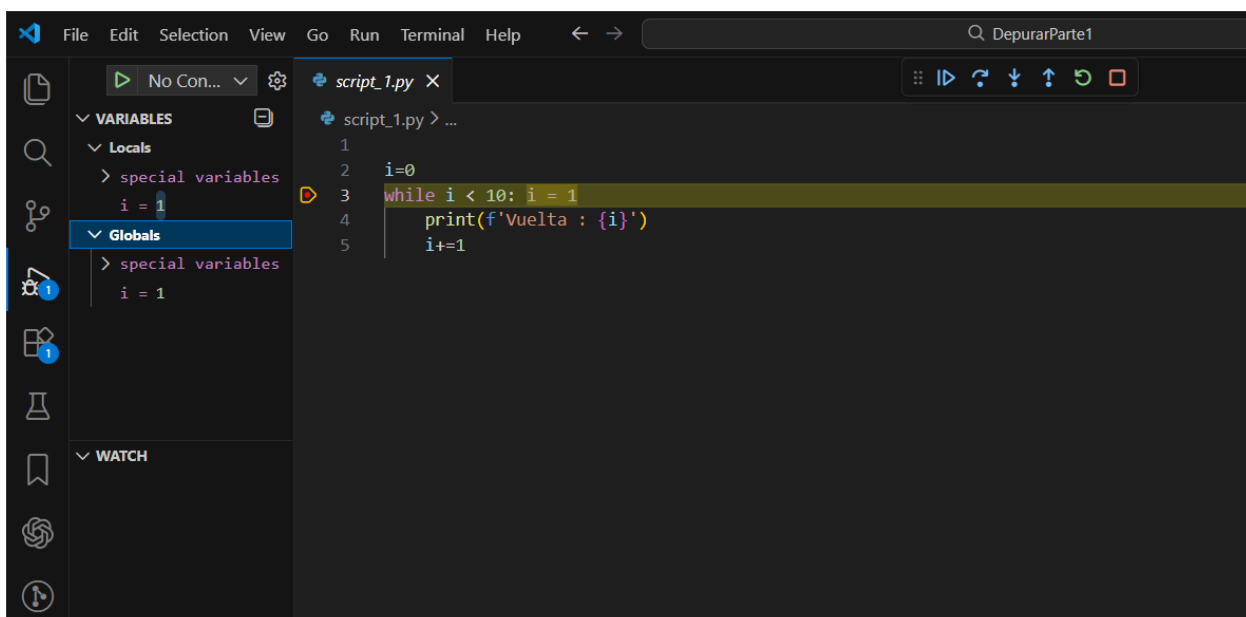
Capturas: Uso de los botones Step Over, Step Into y Continue.
stepover:



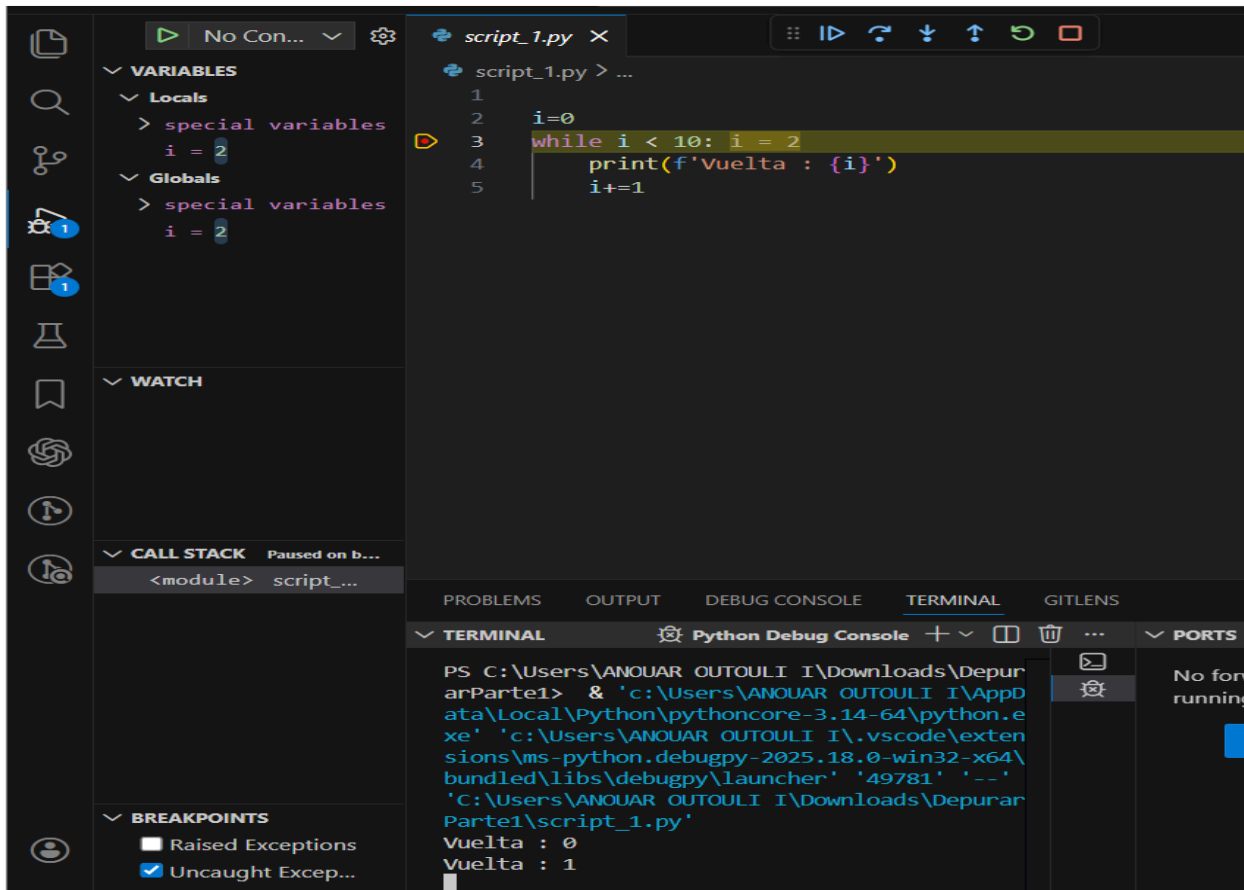
stepInto:



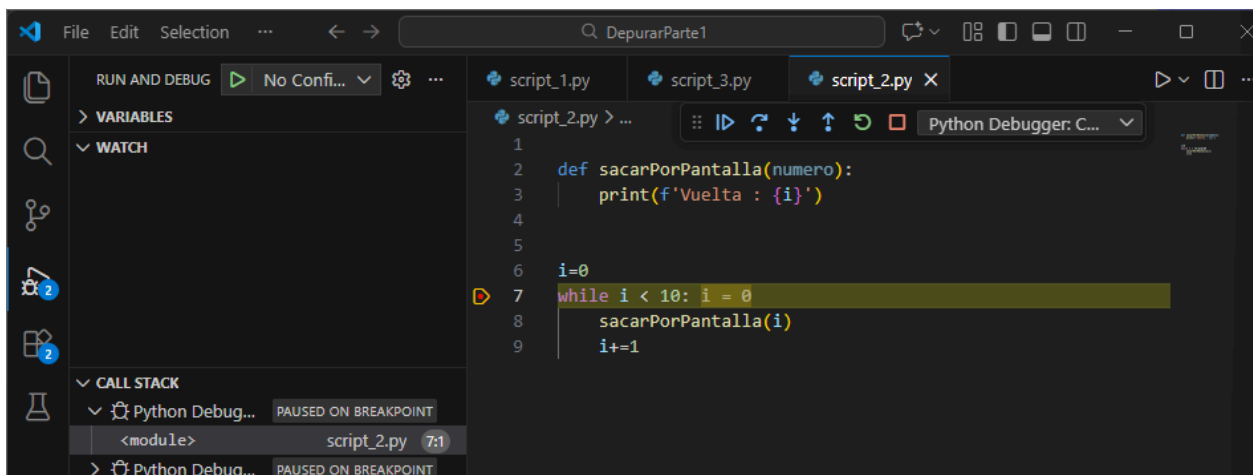
stepout



continue:



Actividad 4.2. Depuración del script_2.py:



Se ha ejecutado el archivo `script_2.py` en modo Debug colocando un punto de ruptura en la llamada a la función.

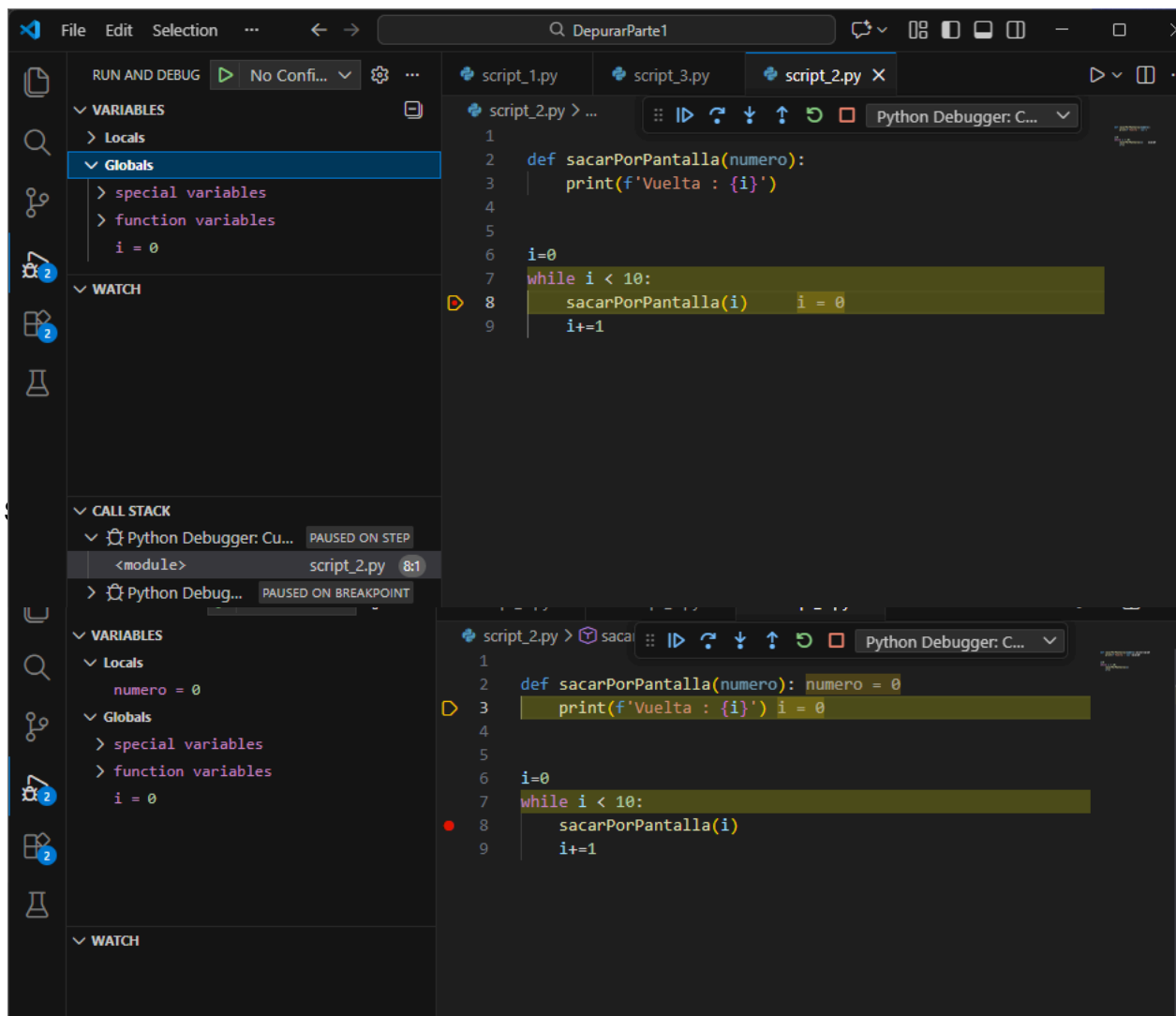
Al utilizar **Step Over**, la función se ejecuta sin entrar en su código interno.

Mediante **Step Into**, el depurador entra dentro de la función `sacarPorPantalla`, permitiendo ejecutar su código línea a línea.

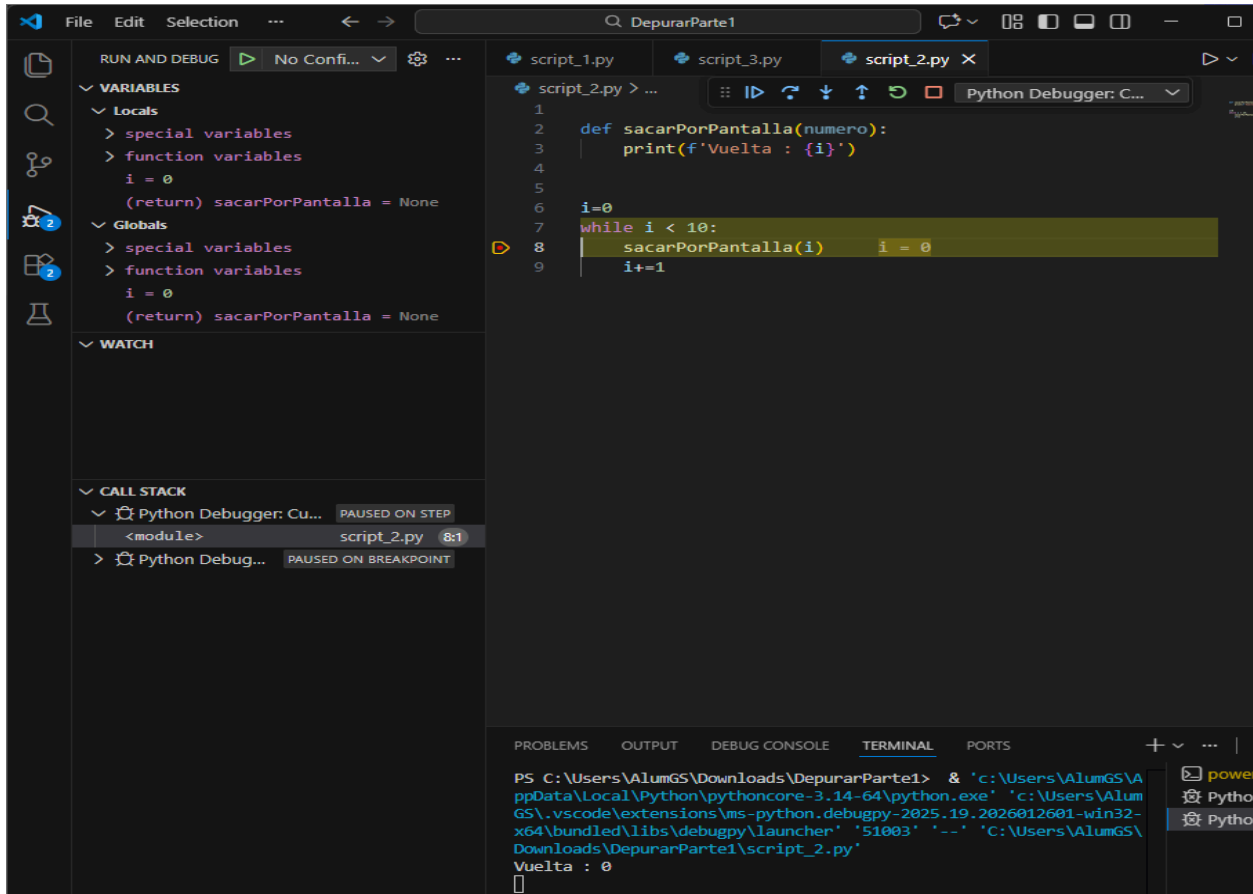
El modo **Step Out** permite salir de la función y continuar la ejecución normal del programa.

Finalmente, con **Continue**, el programa completa su ejecución mostrando el resultado por pantalla.

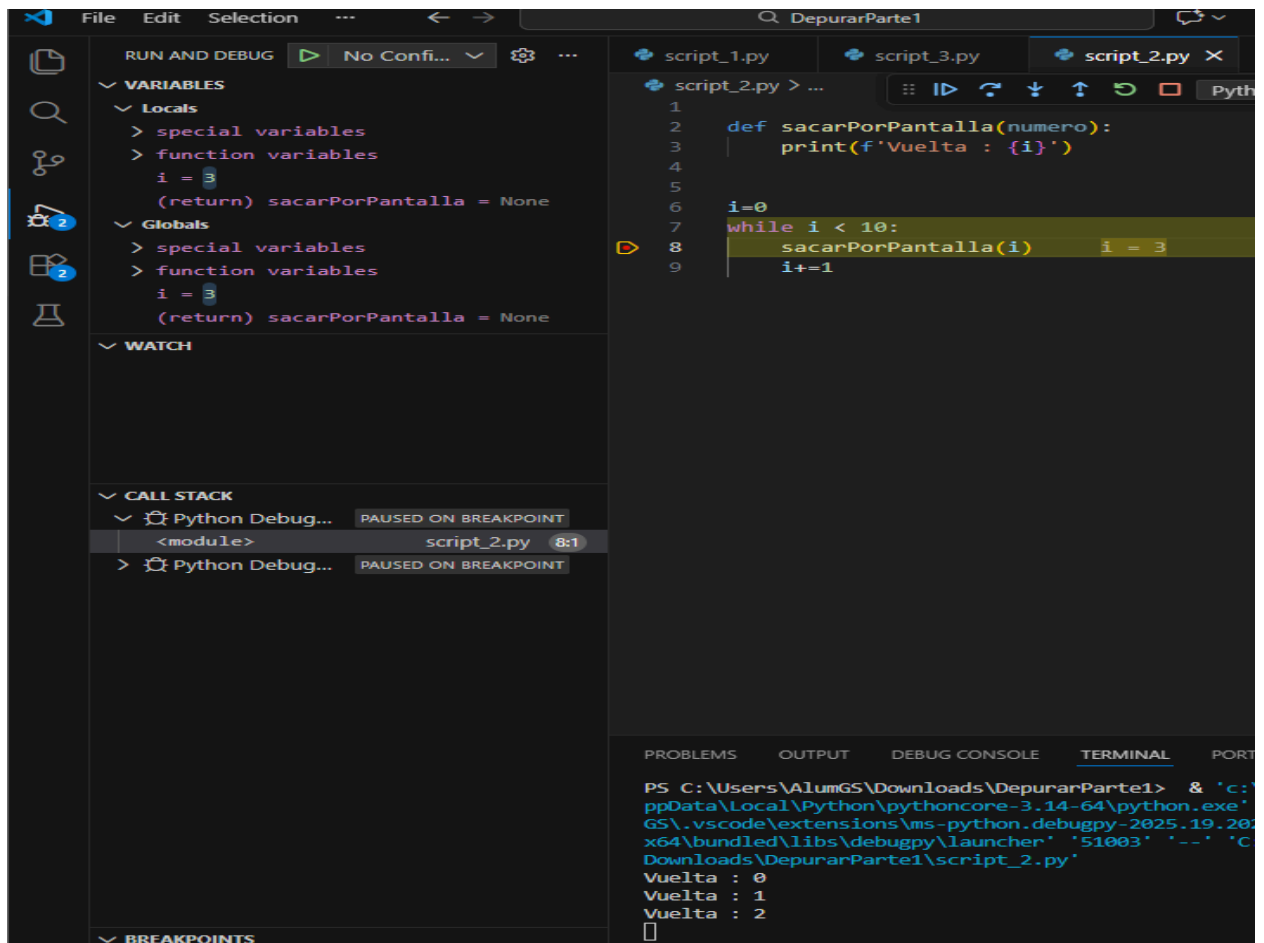
Stepover:



Stepout:



continue:



Actividad 4.3. Depuración del script_3.py

Al ejecutar el archivo `script_3.py` en modo normal, el programa muestra por pantalla los elementos de la lista, pero al final de la ejecución se produce un error de tipo **IndexError**.

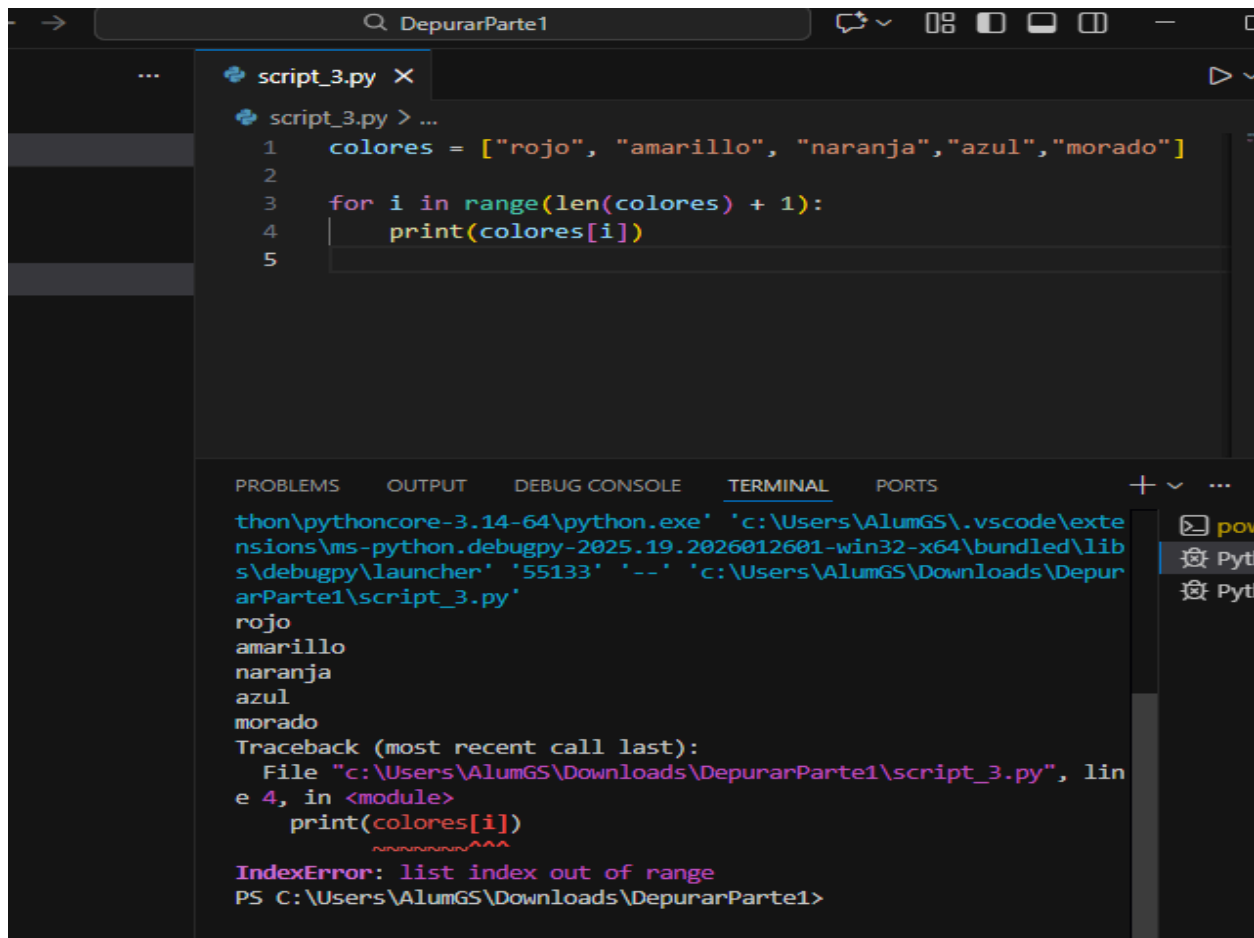
Tal y como se observa en la primera captura, el error aparece porque el bucle `for` está definido como `range(len(colores) + 1)`, lo que provoca que el índice llegue a un valor superior al tamaño de la lista, intentando acceder a una posición inexistente.

Para localizar el error, se ha utilizado el modo **Debug**, colocando un punto de ruptura y ejecutando el programa paso a paso.

Mediante el uso de **Step Over**, se ha podido observar cómo el valor de la variable `i` va aumentando correctamente hasta que alcanza un valor fuera del rango permitido, momento en el que se produce el error.

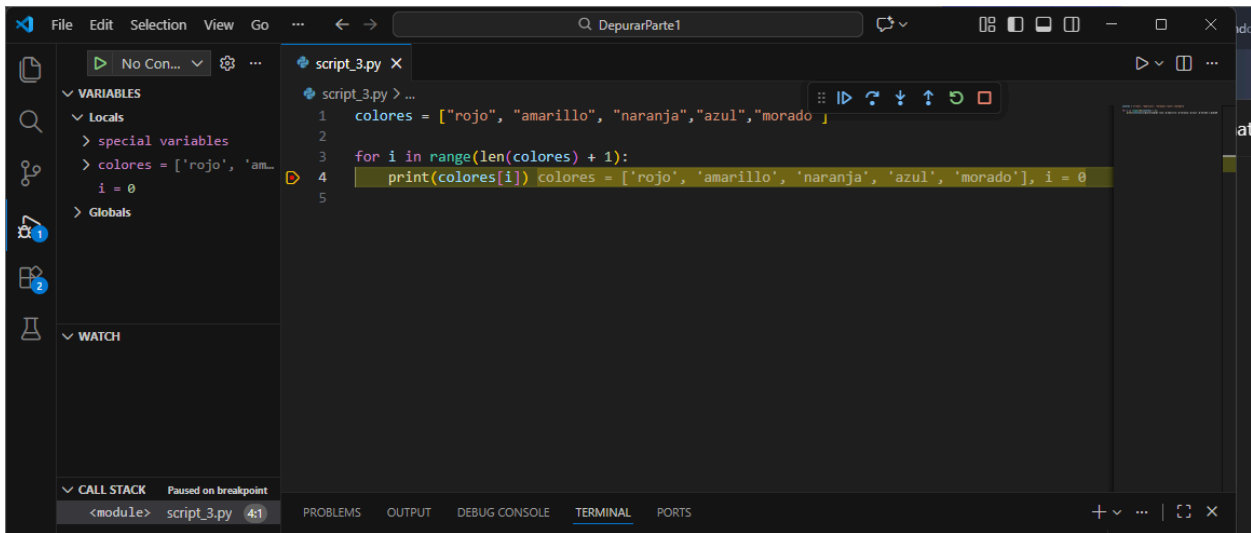
Tras identificar el problema, se ha corregido el código modificando el bucle a `range(len(colores))`, evitando así el acceso fuera de los límites de la lista.

Captura 1: Ejecución del script en modo normal mostrando el error *IndexError*

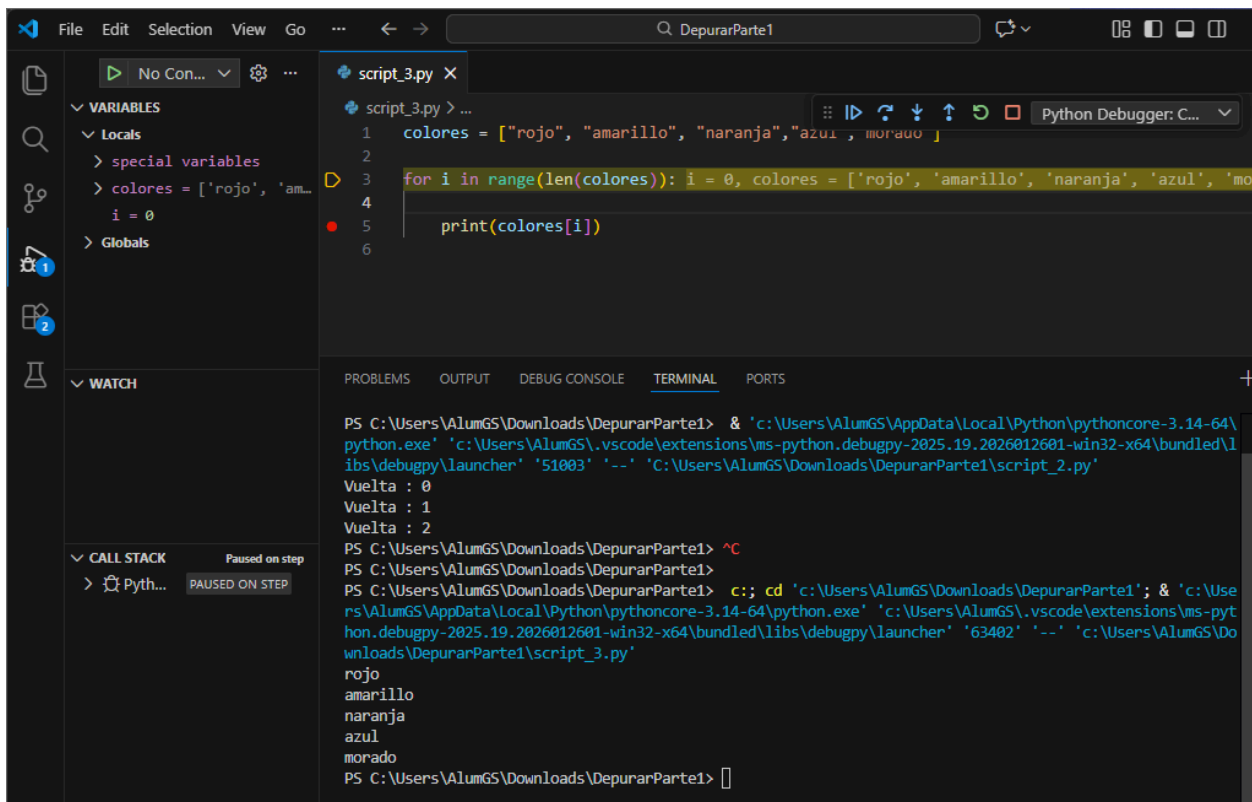


```
DepurarParte1
script_3.py X
script_3.py > ...
1 colores = ["rojo", "amarillo", "naranja","azul","morado"]
2
3 for i in range(len(colores) + 1):
4     print(colores[i])
5

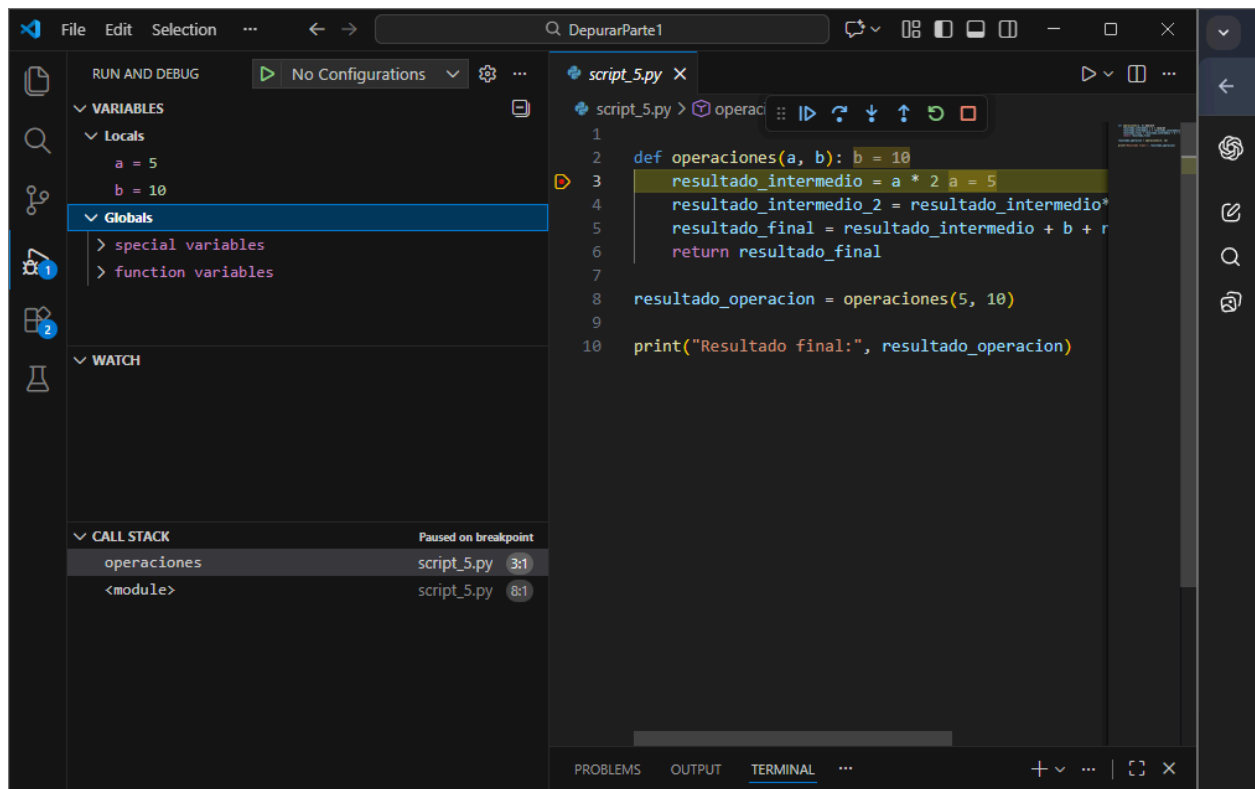
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
thon\pythoncore-3.14-64\python.exe' 'c:\Users\AlumGS\.vscode\extensions\ms-python.debugpy-2025.19.2026012601-win32-x64\bundled\libs\debugpy\launcher' '55133' '--' 'c:\Users\AlumGS\Downloads\DepurarParte1\script_3.py'
rojo
amarillo
naranja
azul
morado
Traceback (most recent call last):
  File "c:\Users\AlumGS\Downloads\DepurarParte1\script_3.py", line 4, in <module>
    print(colores[i])
          ~~~~~^~
IndexError: list index out of range
PS C:\Users\AlumGS\Downloads\DepurarParte1>
```



Captura 2: Uso del modo Debug con punto de ruptura y observación de las variables



Actividad 4.4 – Ejecución del `script_4.py`:



Al ejecutar el archivo `script_4.py`, el programa produce un error durante la ejecución.

El error se debe a que el código intenta realizar una operación no válida entre distintos tipos de datos. En concreto, se intenta concatenar una cadena de texto con un número entero.

Esta situación provoca un error de tipo `TypeError`, ya que Python no permite sumar directamente un valor de tipo `str` con un valor de tipo `int`.

El error se produce durante la ejecución del programa debido a la falta de control de los tipos de datos utilizados.

Actividad 4.5 – Depuración del `script_5.py`:

Se ha ejecutado el archivo `script_5.py` en modo Debug colocando un punto de ruptura dentro de la función.

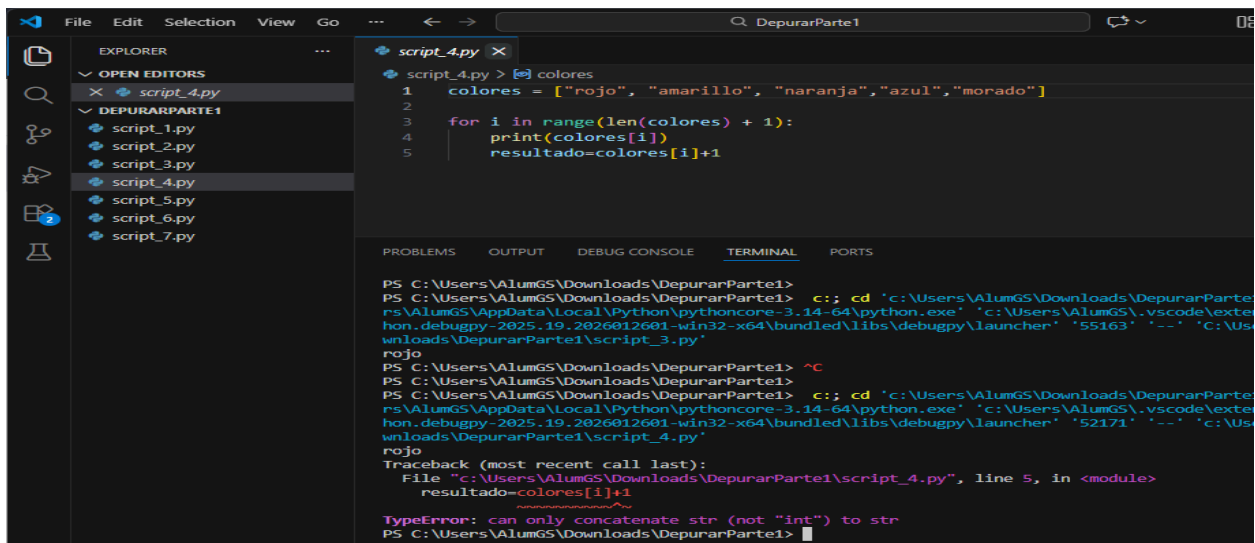
Mediante la ejecución paso a paso, se han observado los valores de las variables y cómo se van actualizando durante el cálculo.

Se ha utilizado **Step Into** para entrar en la función y analizar su ejecución interna, y **Step Out**

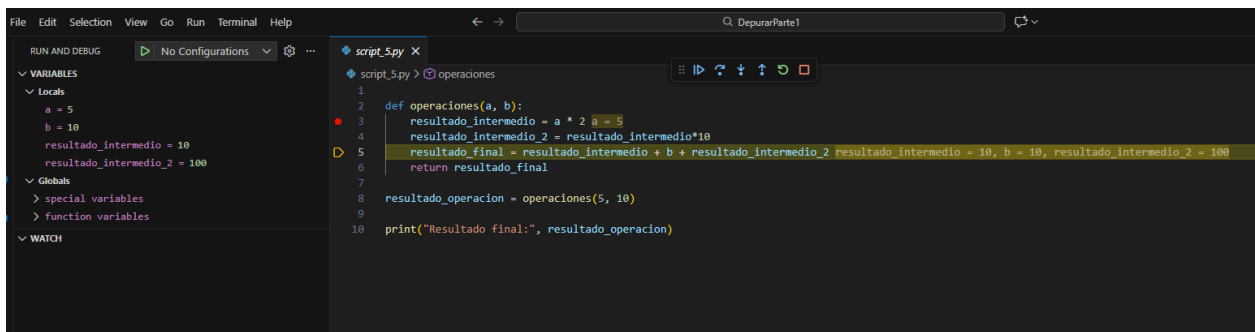
para salir de la función y volver al programa principal.

Finalmente, el programa finaliza correctamente mostrando el resultado final por pantalla.

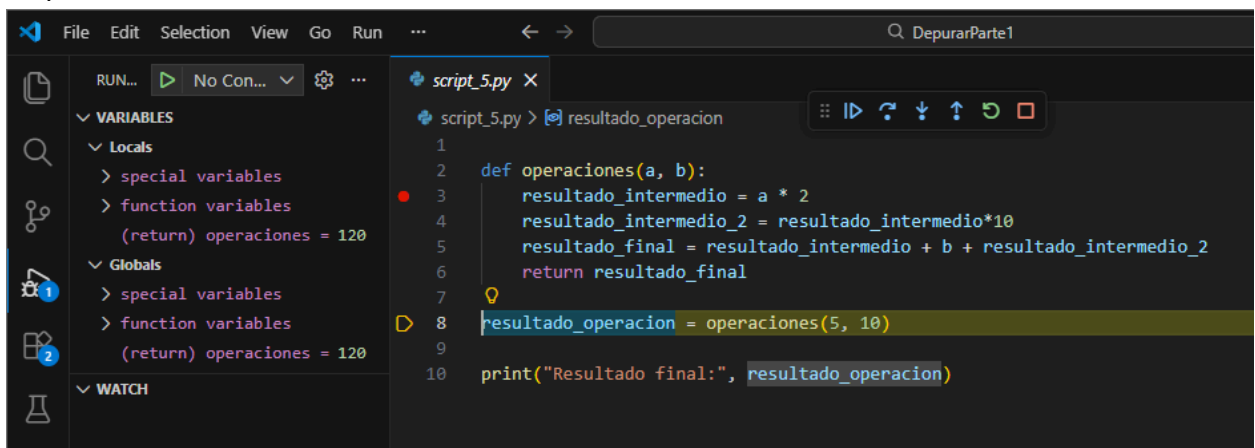
Step over:



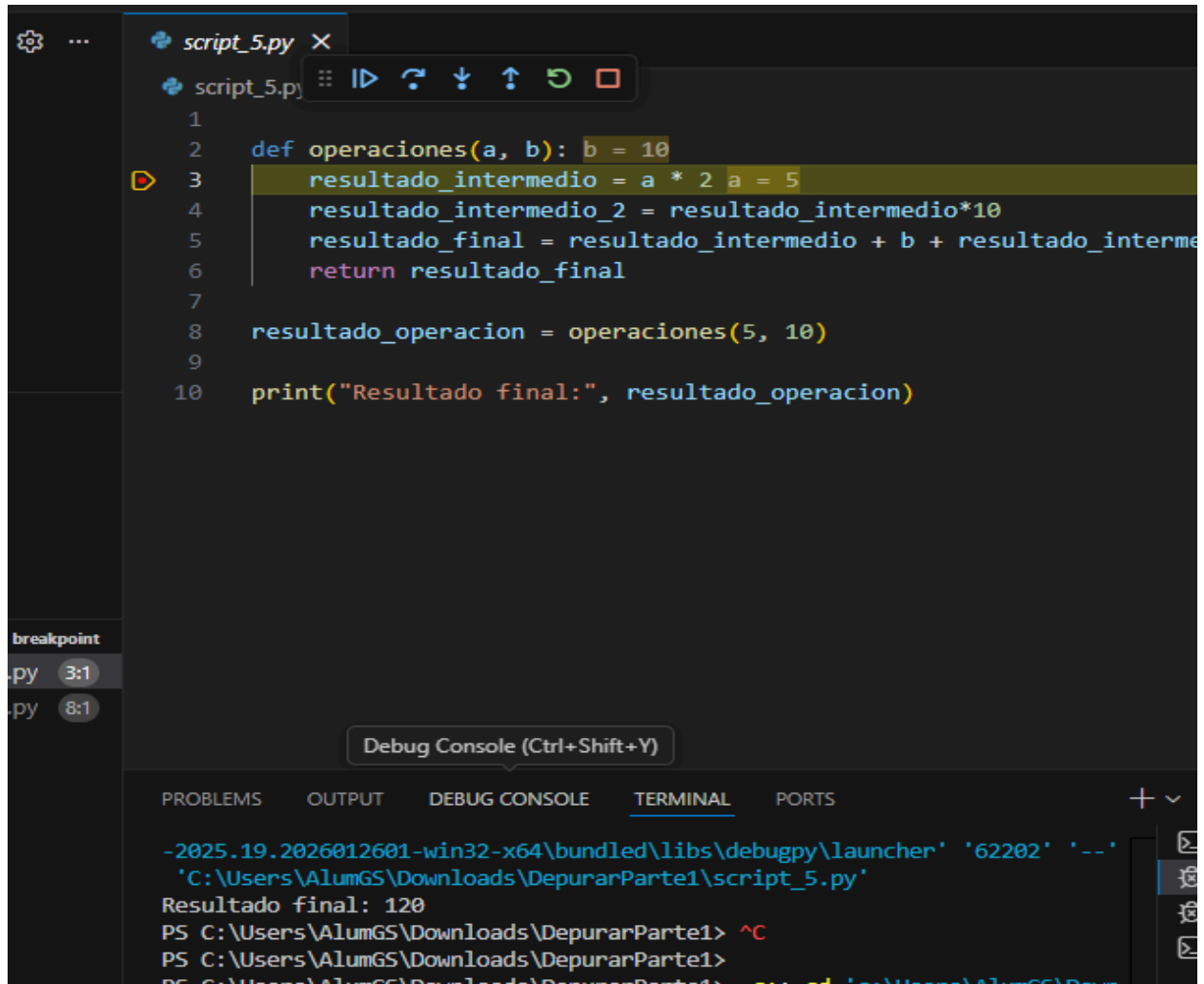
Step Into:



Step out



continue:



```
1
2 def operaciones(a, b): b = 10
3 resultado_intermedio = a * 2 a = 5
4 resultado_intermedio_2 = resultado_intermedio*10
5 resultado_final = resultado_intermedio + b + resultado_intermedio_2
6 return resultado_final
7
8 resultado_operacion = operaciones(5, 10)
9
10 print("Resultado final:", resultado_operacion)
```

breakpoint

py 3:1

py 8:1

Debug Console (Ctrl+Shift+Y)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

-2025.19.2026012601-win32-x64\bundled\libs\debugpy\launcher' '62202' '--' 'C:\Users\AlumGS\Downloads\DepurarParte1\script_5.py'

Resultado final: 120

PS C:\Users\AlumGS\Downloads\DepurarParte1> ^C

PS C:\Users\AlumGS\Downloads\DepurarParte1>

PS C:\Users\AlumGS\Downloads\DepurarParte1> set-ed 'C:\Users\AlumGS\Depur

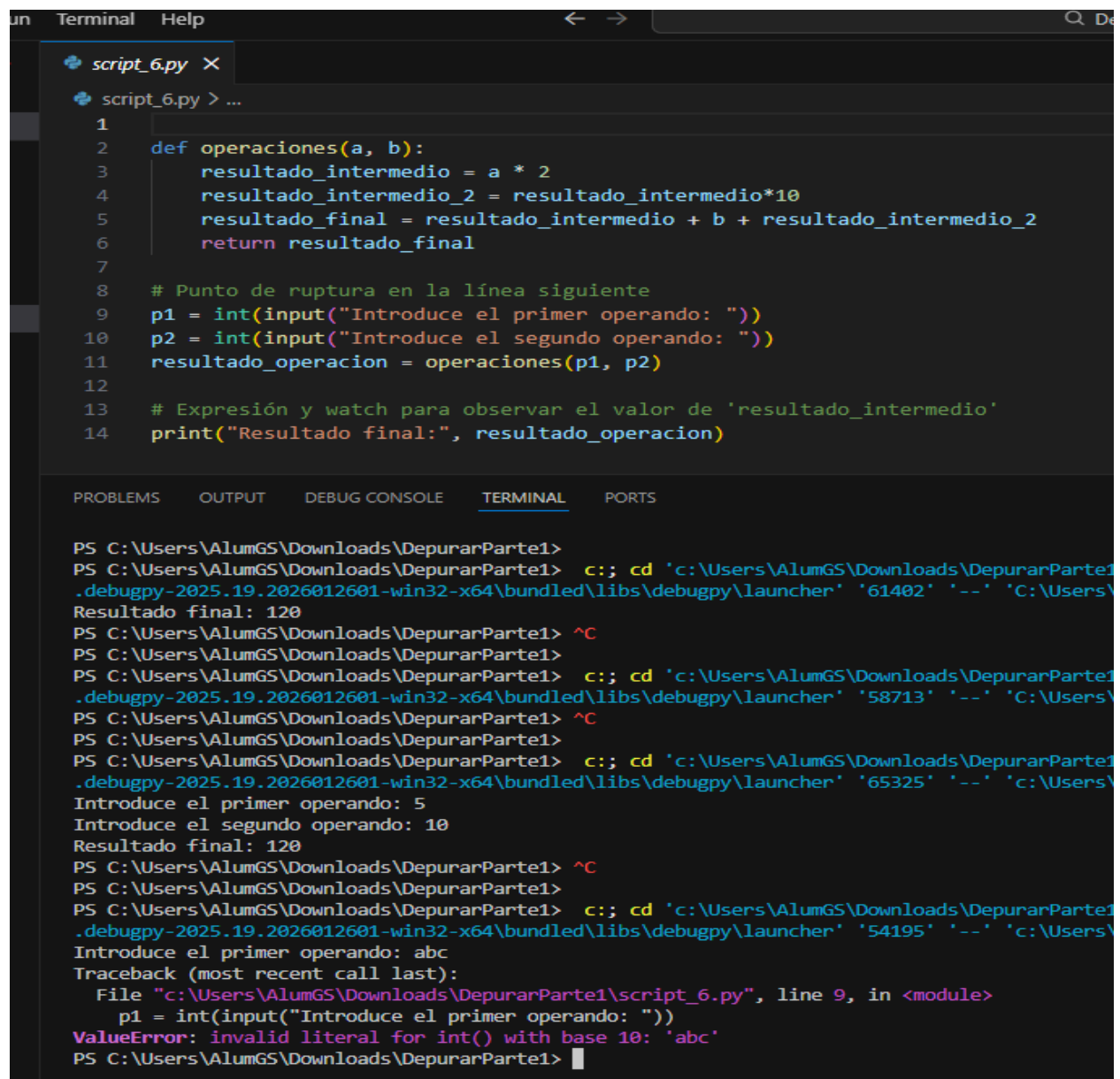
Actividad 4.6:

Se ha ejecutado el archivo `script_6.py` introduciendo valores por teclado.

Cuando se introducen valores numéricos, el programa se ejecuta correctamente y muestra el resultado final.

Sin embargo, al introducir letras en lugar de números, se produce un error de tipo *ValueError*, ya que el programa intenta convertir la entrada a un número entero sin validar los datos introducidos.

Introducir números

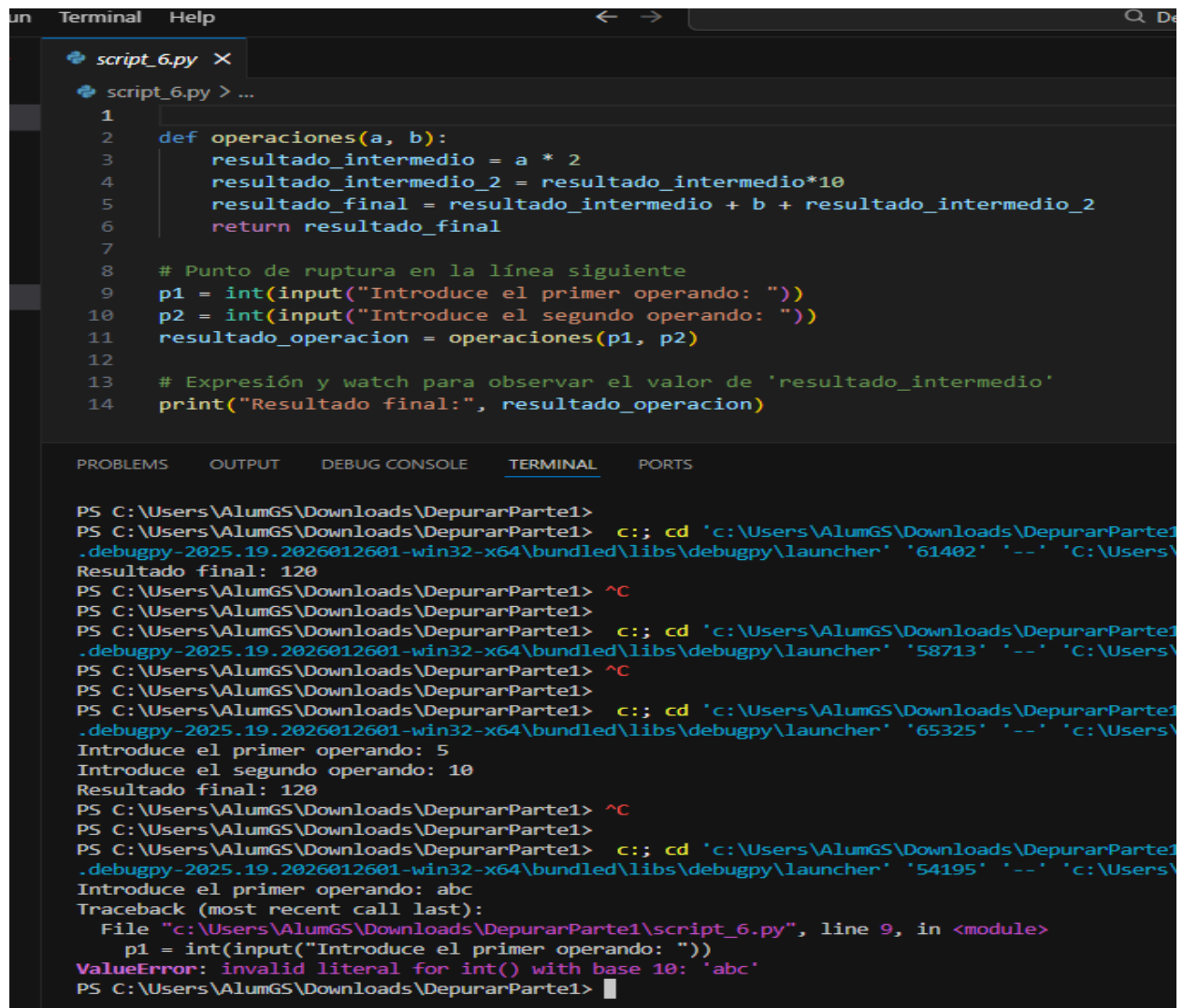


```
un Terminal Help
script_6.py X
script_6.py > ...
1
2 def operaciones(a, b):
3     resultado_intermedio = a * 2
4     resultado_intermedio_2 = resultado_intermedio*10
5     resultado_final = resultado_intermedio + b + resultado_intermedio_2
6     return resultado_final
7
8 # Punto de ruptura en la línea siguiente
9 p1 = int(input("Introduce el primer operando: "))
10 p2 = int(input("Introduce el segundo operando: "))
11 resultado_operacion = operaciones(p1, p2)
12
13 # Expresión y watch para observar el valor de 'resultado_intermedio'
14 print("Resultado final:", resultado_operacion)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\AlumGS\Downloads\DepurarParte1>
PS C:\Users\AlumGS\Downloads\DepurarParte1> c.; cd 'c:\Users\AlumGS\Downloads\DepurarParte1
.debugpy-2025.19.2026012601-win32-x64\bundled\libs\debugpy\launcher' '61402' '--' 'C:\Users\
Resultado final: 120
PS C:\Users\AlumGS\Downloads\DepurarParte1> ^C
PS C:\Users\AlumGS\Downloads\DepurarParte1>
PS C:\Users\AlumGS\Downloads\DepurarParte1> c.; cd 'c:\Users\AlumGS\Downloads\DepurarParte1
.debugpy-2025.19.2026012601-win32-x64\bundled\libs\debugpy\launcher' '58713' '--' 'C:\Users\
PS C:\Users\AlumGS\Downloads\DepurarParte1> ^C
PS C:\Users\AlumGS\Downloads\DepurarParte1>
PS C:\Users\AlumGS\Downloads\DepurarParte1> c.; cd 'c:\Users\AlumGS\Downloads\DepurarParte1
.debugpy-2025.19.2026012601-win32-x64\bundled\libs\debugpy\launcher' '65325' '--' 'C:\Users\
Introduce el primer operando: 5
Introduce el segundo operando: 10
Resultado final: 120
PS C:\Users\AlumGS\Downloads\DepurarParte1> ^C
PS C:\Users\AlumGS\Downloads\DepurarParte1>
PS C:\Users\AlumGS\Downloads\DepurarParte1> c.; cd 'c:\Users\AlumGS\Downloads\DepurarParte1
.debugpy-2025.19.2026012601-win32-x64\bundled\libs\debugpy\launcher' '54195' '--' 'C:\Users\
Introduce el primer operando: abc
Traceback (most recent call last):
  File "c:\Users\AlumGS\Downloads\DepurarParte1\script_6.py", line 9, in <module>
    p1 = int(input("Introduce el primer operando: "))
ValueError: invalid literal for int() with base 10: 'abc'
PS C:\Users\AlumGS\Downloads\DepurarParte1> |
```

Introducir Letras:



The image shows a Visual Studio Code editor window with a Python script named `script_6.py` and its execution output in the terminal.

Script Code:

```
1
2 def operaciones(a, b):
3     resultado_intermedio = a * 2
4     resultado_intermedio_2 = resultado_intermedio*10
5     resultado_final = resultado_intermedio + b + resultado_intermedio_2
6     return resultado_final
7
8 # Punto de ruptura en la línea siguiente
9 p1 = int(input("Introduce el primer operando: "))
10 p2 = int(input("Introduce el segundo operando: "))
11 resultado_operacion = operaciones(p1, p2)
12
13 # Expresión y watch para observar el valor de 'resultado_intermedio'
14 print("Resultado final:", resultado_operacion)
```

Terminal Output:

```
PS C:\Users\AlumGS\Downloads\DepurarParte1>
PS C:\Users\AlumGS\Downloads\DepurarParte1> c;; cd 'c:\Users\AlumGS\Downloads\DepurarParte1\
.debugpy-2025.19.2026012601-win32-x64\bundled\libs\debugpy\launcher' '61402' '--' 'c:\Users\
Resultado final: 120
PS C:\Users\AlumGS\Downloads\DepurarParte1> ^C
PS C:\Users\AlumGS\Downloads\DepurarParte1>
PS C:\Users\AlumGS\Downloads\DepurarParte1> c;; cd 'c:\Users\AlumGS\Downloads\DepurarParte1\
.debugpy-2025.19.2026012601-win32-x64\bundled\libs\debugpy\launcher' '58713' '--' 'c:\Users\
PS C:\Users\AlumGS\Downloads\DepurarParte1> ^C
PS C:\Users\AlumGS\Downloads\DepurarParte1>
PS C:\Users\AlumGS\Downloads\DepurarParte1> c;; cd 'c:\Users\AlumGS\Downloads\DepurarParte1\
.debugpy-2025.19.2026012601-win32-x64\bundled\libs\debugpy\launcher' '65325' '--' 'c:\Users\
Introduce el primer operando: 5
Introduce el segundo operando: 10
Resultado final: 120
PS C:\Users\AlumGS\Downloads\DepurarParte1> ^C
PS C:\Users\AlumGS\Downloads\DepurarParte1>
PS C:\Users\AlumGS\Downloads\DepurarParte1> c;; cd 'c:\Users\AlumGS\Downloads\DepurarParte1\
.debugpy-2025.19.2026012601-win32-x64\bundled\libs\debugpy\launcher' '54195' '--' 'c:\Users\
Introduce el primer operando: abc
Traceback (most recent call last):
  File "c:\Users\AlumGS\Downloads\DepurarParte1\script_6.py", line 9, in <module>
    p1 = int(input("Introduce el primer operando: "))
ValueError: invalid literal for int() with base 10: 'abc'
PS C:\Users\AlumGS\Downloads\DepurarParte1> █
```

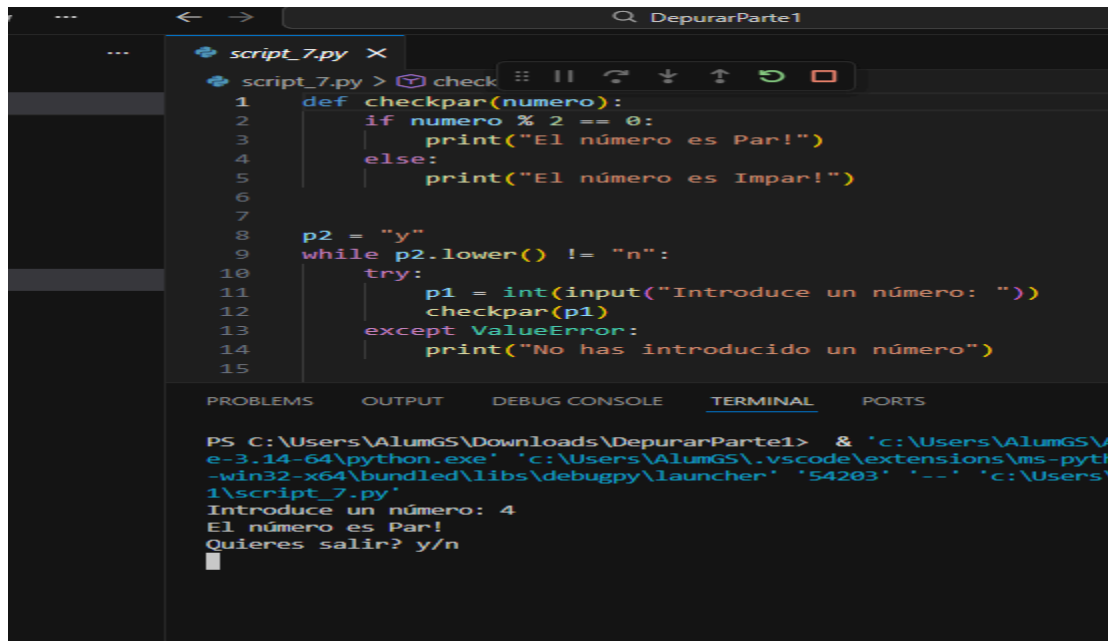
Actividad 4.7 – Ejecución del `script_7.py`:

Se ha ejecutado el archivo `script_7.py` para comprobar su funcionamiento.

El programa determina correctamente si un número es par o impar y permite continuar o finalizar la ejecución según la opción introducida por el usuario.

Se ha comprobado que el programa gestiona correctamente tanto letras mayúsculas como minúsculas en la opción de salida, gracias al uso del método `lower()`.

Además, el uso de `try/except` evita que el programa se detenga cuando el usuario introduce un valor no numérico

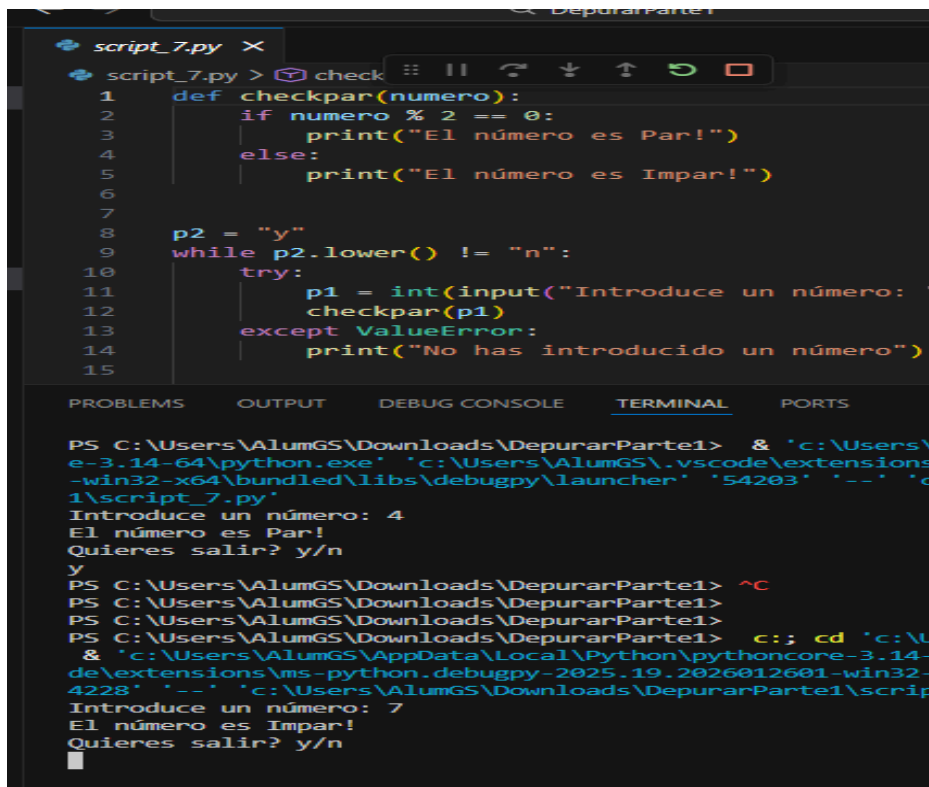


The screenshot shows a VS Code editor window with a file named `script_7.py`. The code defines a `checkpar` function that checks if a number is even or odd. It then enters a loop where it asks the user to input a number and checks it. If the input is not a number, it catches a `ValueError` and prints a message. The terminal output shows the program running and the user entering the number 4, which is correctly identified as even.

```
1 def checkpar(numero):
2     if numero % 2 == 0:
3         print("El número es Par!")
4     else:
5         print("El número es Impar!")
6
7
8 p2 = "y"
9 while p2.lower() != "n":
10    try:
11        p1 = int(input("Introduce un número: "))
12        checkpar(p1)
13    except ValueError:
14        print("No has introducido un número")
15
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\AlumGS\Downloads\DepurarParte1> & 'c:\Users\AlumGS\AppData\Local\Python\pythoncore-3.14-64\python.exe' 'c:\Users\AlumGS\.vscode\extensions\ms-python-win32-x64\bundled\libs\debugpy\launcher' '54203' '--' 'c:\Users\AlumGS\Downloads\DepurarParte1\script_7.py'
Introduce un número: 4
El número es Par!
Quieres salir? y/n
```

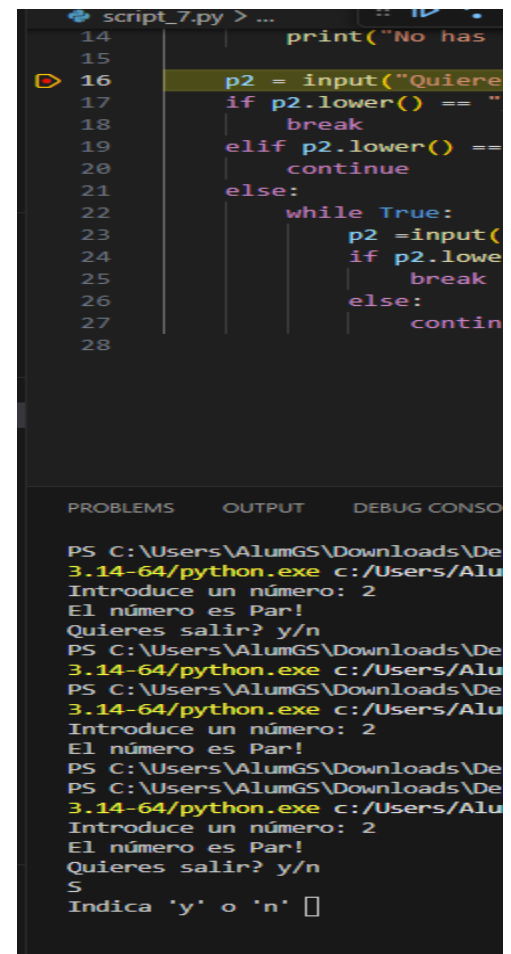


This screenshot shows the same VS Code editor window, but the terminal output continues. The user enters 'y' to continue the loop, then enters the number 7, which is correctly identified as odd. The user then enters 'n' to exit the loop.

```
1 def checkpar(numero):
2     if numero % 2 == 0:
3         print("El número es Par!")
4     else:
5         print("El número es Impar!")
6
7
8 p2 = "y"
9 while p2.lower() != "n":
10    try:
11        p1 = int(input("Introduce un número: "))
12        checkpar(p1)
13    except ValueError:
14        print("No has introducido un número")
15
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\AlumGS\Downloads\DepurarParte1> & 'c:\Users\AlumGS\AppData\Local\Python\pythoncore-3.14-64\python.exe' 'c:\Users\AlumGS\.vscode\extensions\ms-python-win32-x64\bundled\libs\debugpy\launcher' '54203' '--' 'c:\Users\AlumGS\Downloads\DepurarParte1\script_7.py'
Introduce un número: 4
El número es Par!
Quieres salir? y/n
y
PS C:\Users\AlumGS\Downloads\DepurarParte1> ^C
PS C:\Users\AlumGS\Downloads\DepurarParte1>
PS C:\Users\AlumGS\Downloads\DepurarParte1>
PS C:\Users\AlumGS\Downloads\DepurarParte1> c:; cd 'c:\Users\AlumGS\Downloads\DepurarParte1'
PS C:\Users\AlumGS\Downloads\DepurarParte1> & 'c:\Users\AlumGS\AppData\Local\Python\pythoncore-3.14-64\python.exe' 'c:\Users\AlumGS\.vscode\extensions\ms-python-win32-x64\bundled\libs\debugpy\launcher' '54203' '--' 'c:\Users\AlumGS\Downloads\DepurarParte1\script_7.py'
Introduce un número: 7
El número es Impar!
Quieres salir? y/n
```



This screenshot shows the same VS Code editor window, but the code is modified to include a loop that asks the user if they want to continue. The terminal output shows the program running and the user entering the number 2, which is correctly identified as even. The user then enters 'y' to continue the loop, and the program asks if they want to continue. The user enters 'S', which is not 'y' or 'n', so the program prints a message asking the user to indicate 'y' or 'n'.

```
14 print("No has introducido un número")
15
16 p2 = input("Quieres salir? y/n: ")
17 if p2.lower() == "y":
18     break
19 elif p2.lower() == "n":
20     continue
21 else:
22     while True:
23         p2 = input("Indica 'y' o 'n': ")
24         if p2.lower() == "y":
25             break
26         else:
27             continue
28
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

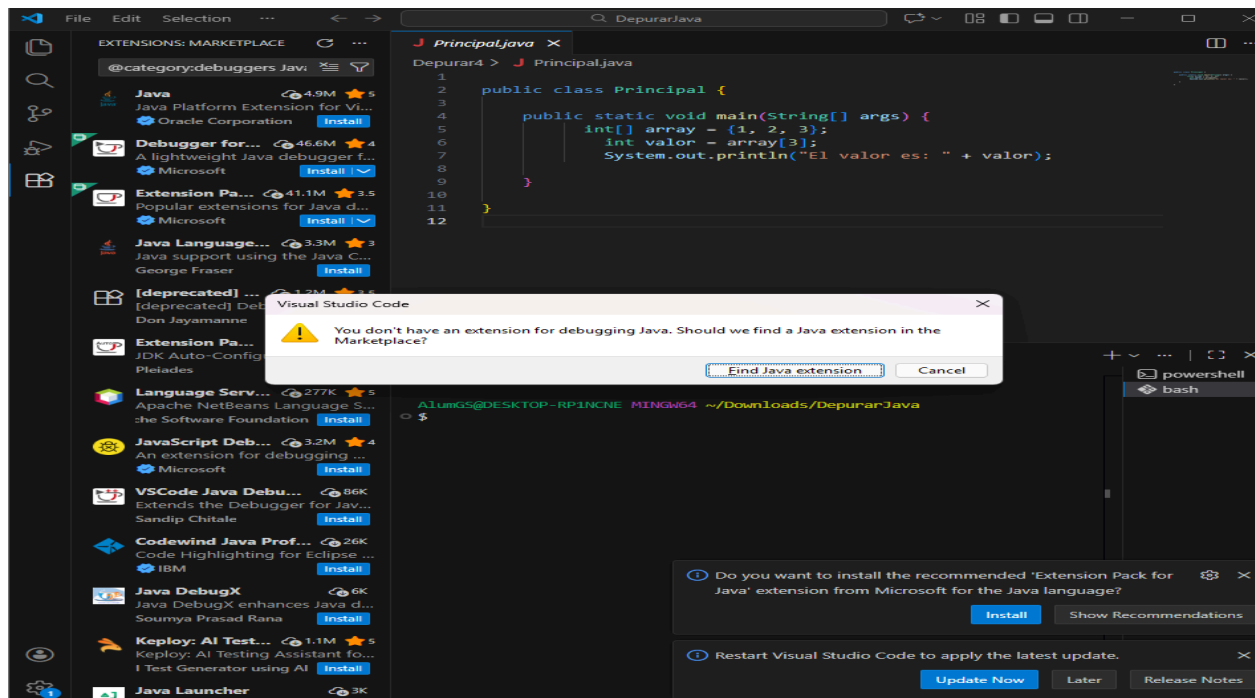
```
PS C:\Users\AlumGS\Downloads\DepurarParte1> & 'c:\Users\AlumGS\AppData\Local\Python\pythoncore-3.14-64\python.exe' 'c:\Users\AlumGS\.vscode\extensions\ms-python-win32-x64\bundled\libs\debugpy\launcher' '54203' '--' 'c:\Users\AlumGS\Downloads\DepurarParte1\script_7.py'
Introduce un número: 2
El número es Par!
Quieres salir? y/n
y
PS C:\Users\AlumGS\Downloads\DepurarParte1> & 'c:\Users\AlumGS\AppData\Local\Python\pythoncore-3.14-64\python.exe' 'c:\Users\AlumGS\.vscode\extensions\ms-python-win32-x64\bundled\libs\debugpy\launcher' '54203' '--' 'c:\Users\AlumGS\Downloads\DepurarParte1\script_7.py'
Introduce un número: 2
El número es Par!
Quieres salir? y/n
S
Indica 'y' o 'n'
```

Actividad 5. Configuración del IDE para otro lenguaje: **Java**:

Se ha intentado ejecutar el archivo `Principal.java` desde Visual Studio Code, siendo necesario instalar el JDK y las extensiones de Java.

Tras la instalación del **Extension Pack for Java**, ha sido posible ejecutar y depurar correctamente el programa.

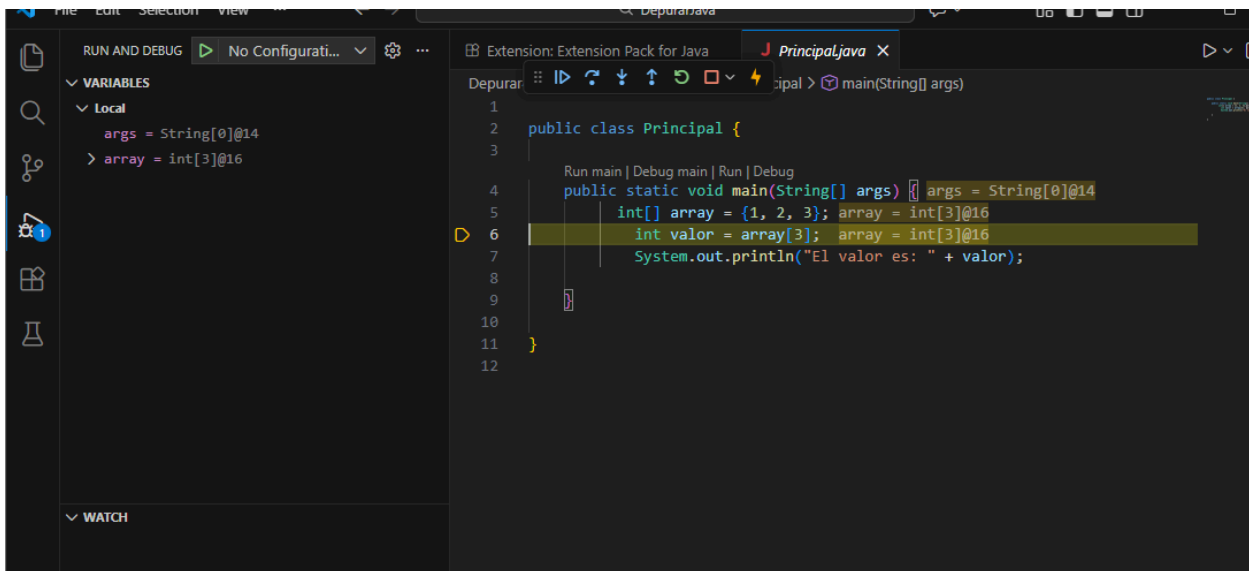
Además, se ha comprobado la instalación de Java mediante el comando `java -version`, confirmando que el entorno está correctamente configurado.



Terminal

```
PS C:\Users\AlumGS\Downloads\DepurarJava> java --version
java 25.0.1 2025-10-21 LTS
Java(TM) SE Runtime Environment (build 25.0.1+8-LTS-27)
Java HotSpot(TM) 64-Bit Server VM (build 25.0.1+8-LTS-27, mixed mode,
sharing)
PS C:\Users\AlumGS\Downloads\DepurarJava> 
```

Step Over:

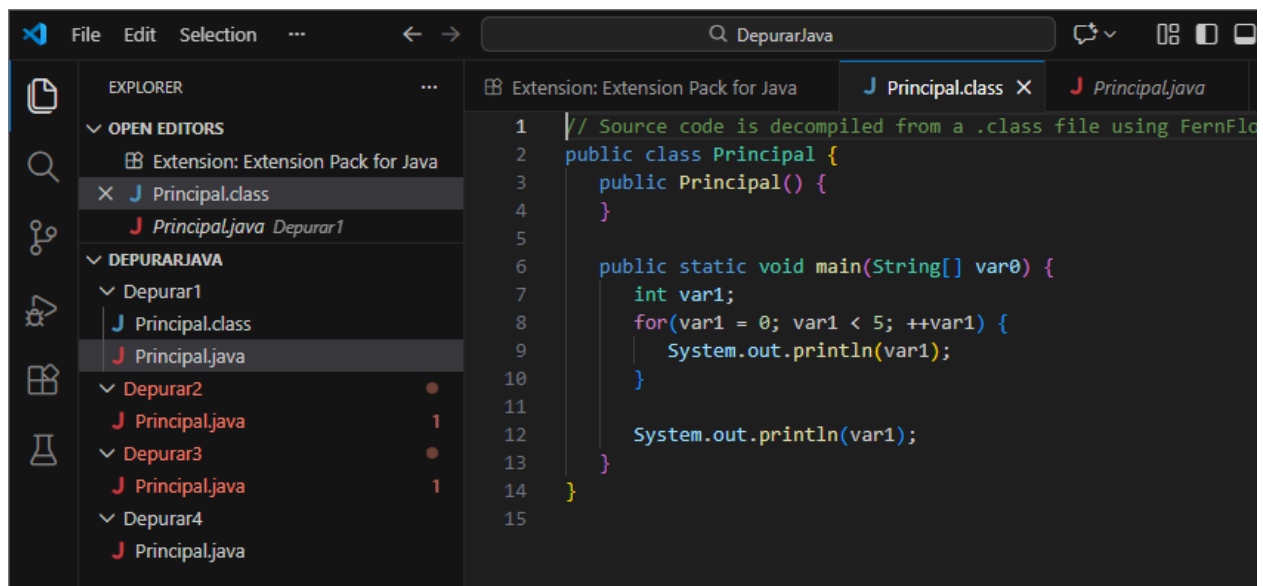


Actividad 6. Compilación y generación del Bytecode (Java):

Se ha compilado el archivo `Principal.java` mediante el comando `javac Principal.java`, generando el archivo `Principal.class`.

El archivo `.class` corresponde al bytecode de Java y no es legible directamente, ya que está destinado a ser ejecutado por la máquina virtual de Java (JVM).

Esto demuestra que Java es un lenguaje compilado, tal y como se ha visto en teoría.

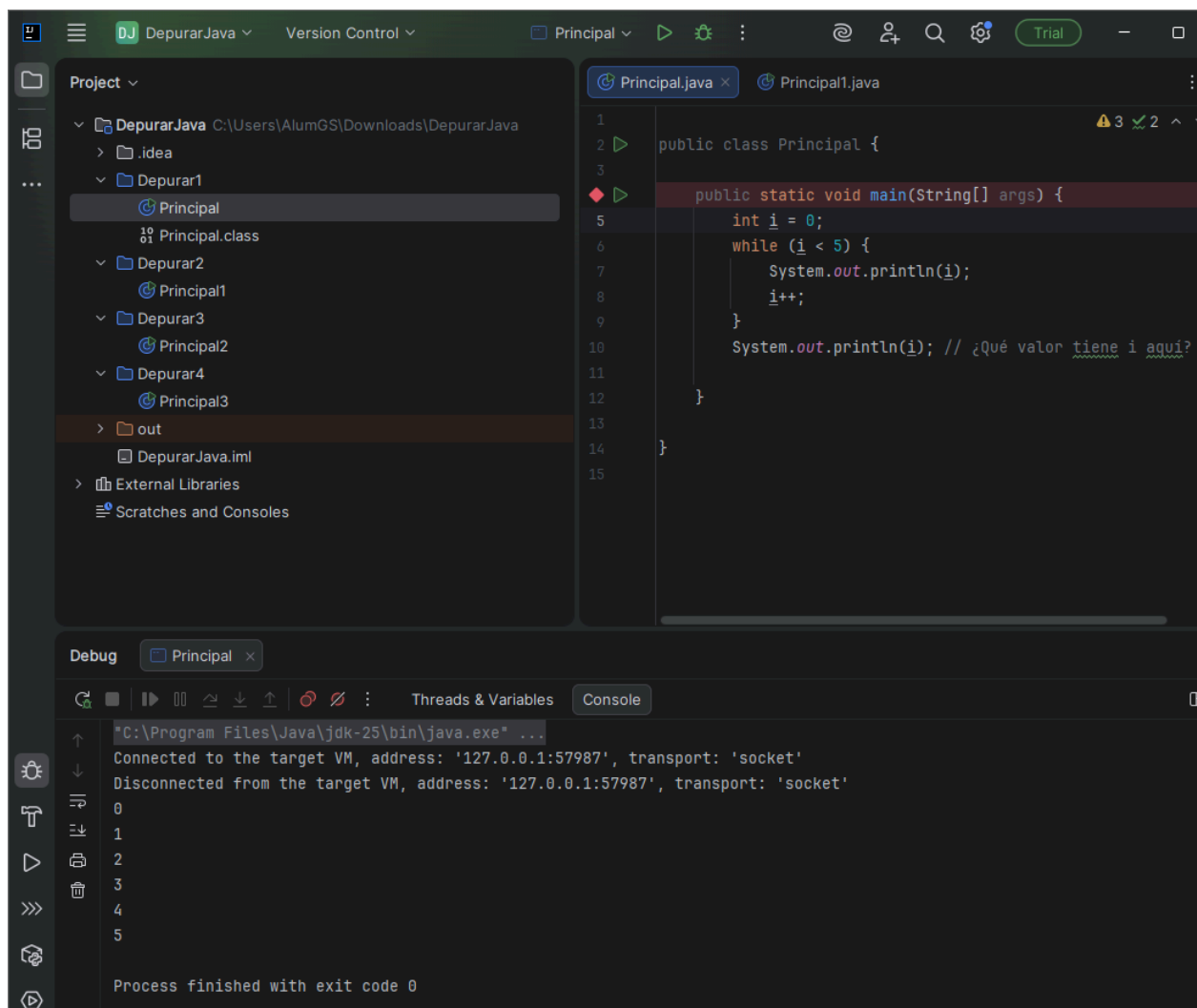


Actividad 6 – Uso del IDE IntelliJ IDEA:

El programa se ha ejecutado correctamente en modo Debug utilizando IntelliJ IDEA.

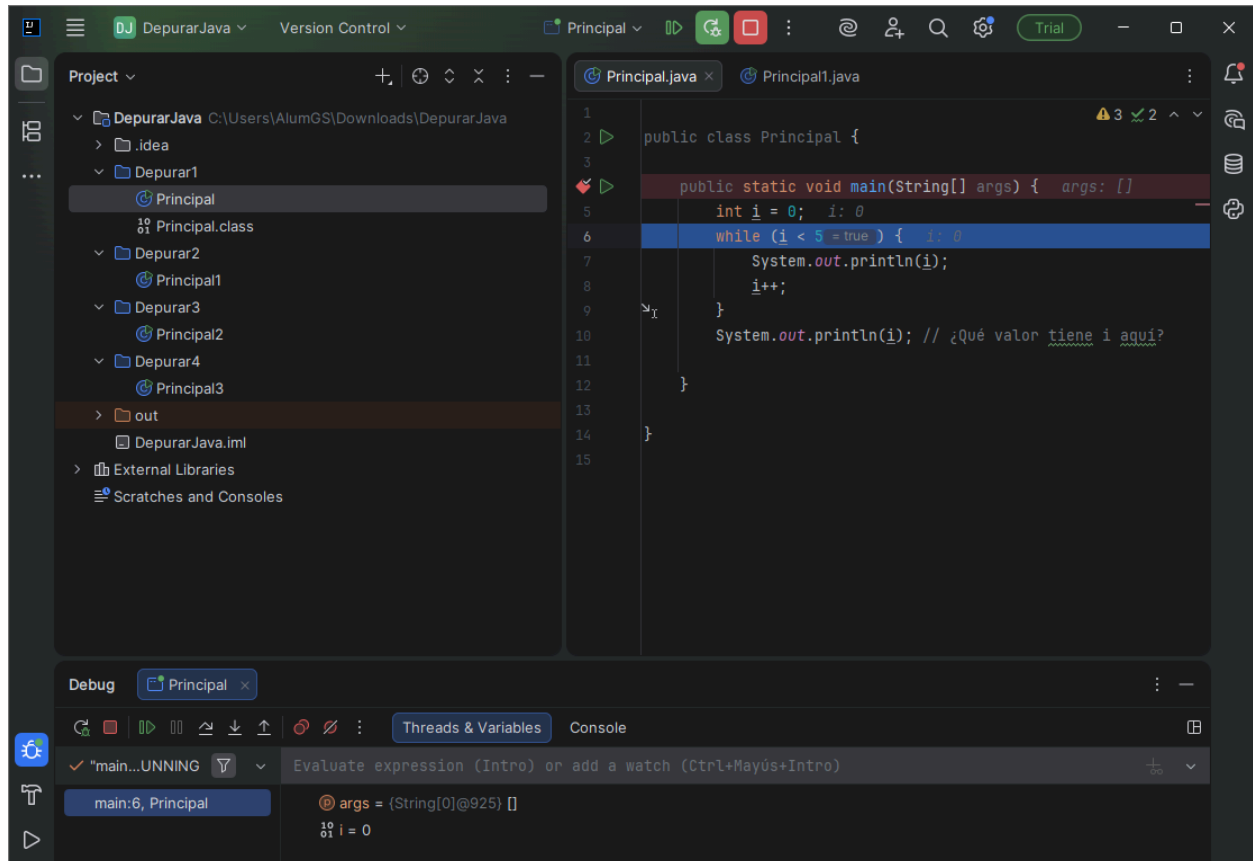
Durante la ejecución del bucle `while`, la variable `i` toma valores de 0 a 4.

Al salir del bucle, el valor de la variable `i` es 5, lo que se comprueba con la impresión final por pantalla.



Se ha utilizado el depurador para ejecutar el programa paso a paso.

Mediante la opción **Step Over**, se ha podido observar el valor de la variable `i` en cada iteración del bucle desde la ventana *Threads & Variables*.



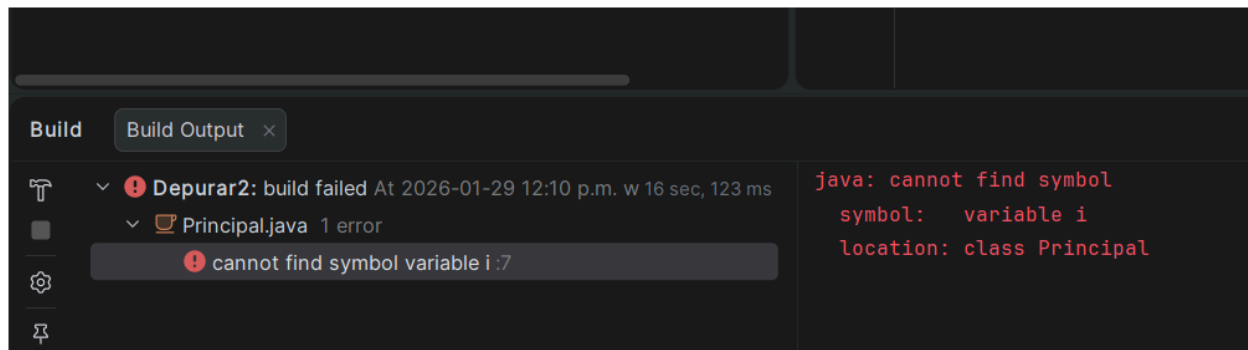
Actividad 7:

El programa devuelve un error porque la variable `i` está declarada dentro del bucle `for` y se intenta usar fuera de él.

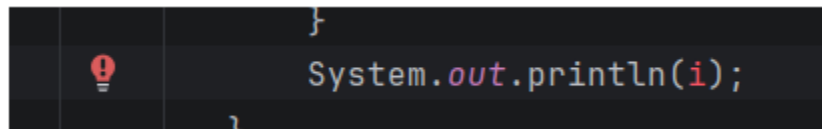
Al comentar la línea que provoca el error, el programa se puede ejecutar en modo depuración.

Utilizando **Step Over**, se puede observar el valor de la variable `i` en cada iteración desde la ventana **Threads & Variables**.

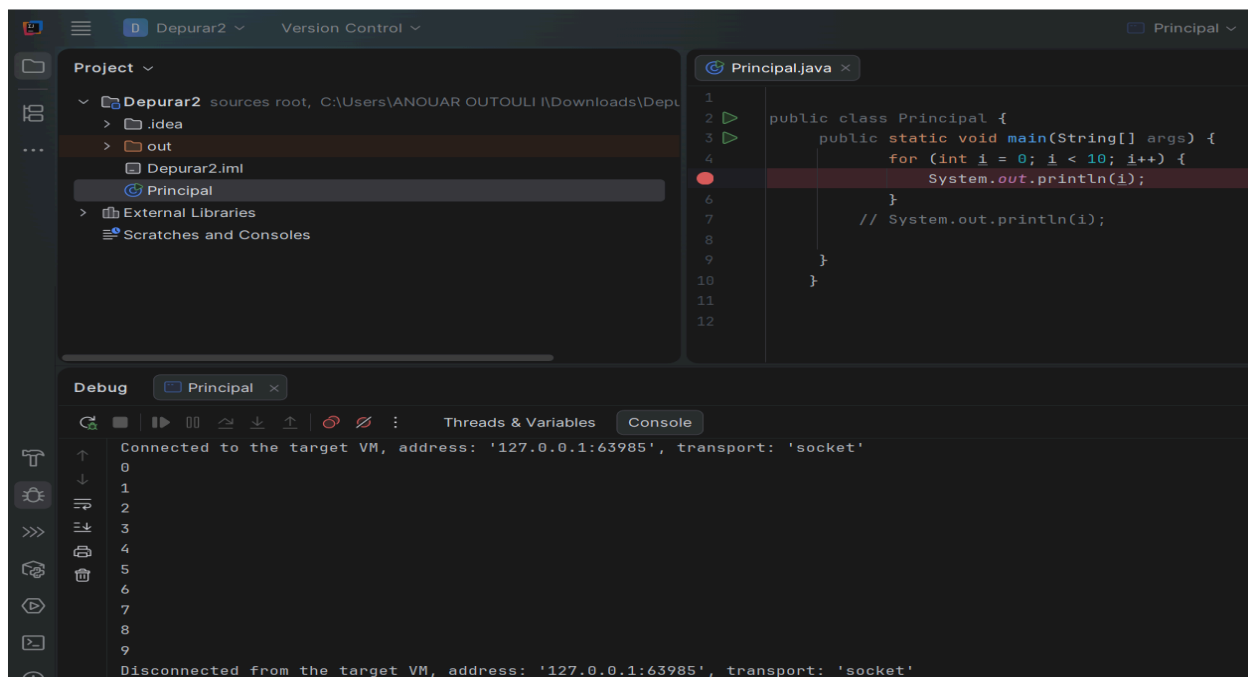
Error:



la línea que da el problema :

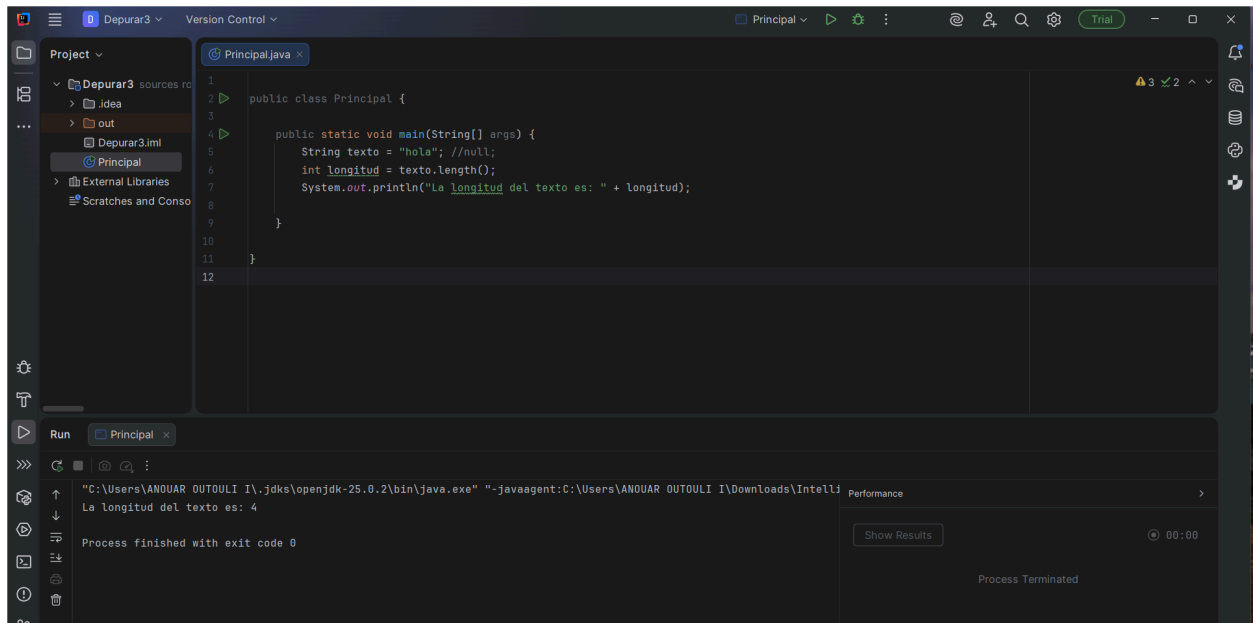


Solución:



Actividad 7 (Depurar3):

En la carpeta **Depurar3**, el programa se ejecuta correctamente y no devuelve ningún error. Por tanto, no es necesario comentar ninguna línea ni realizar depuración, ya que el código funciona de forma correcta.

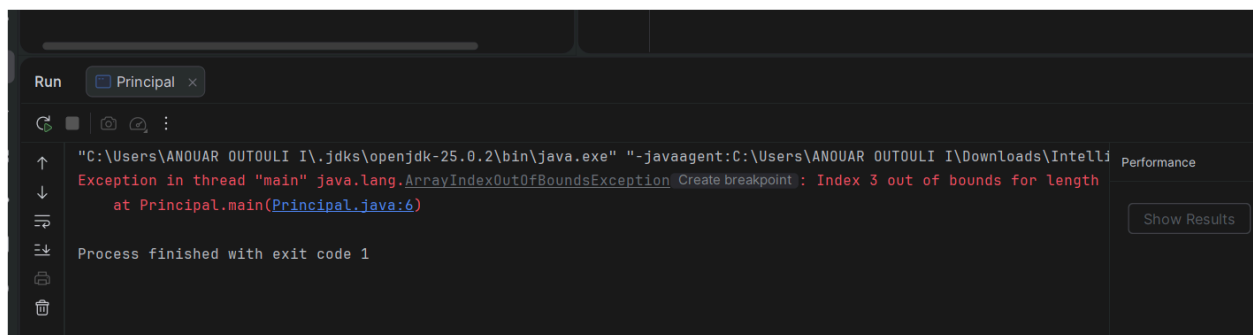


Actividad 8:

El error **ArrayIndexOutOfBoundsException** se produce porque se intenta acceder a una posición del array que no existe.

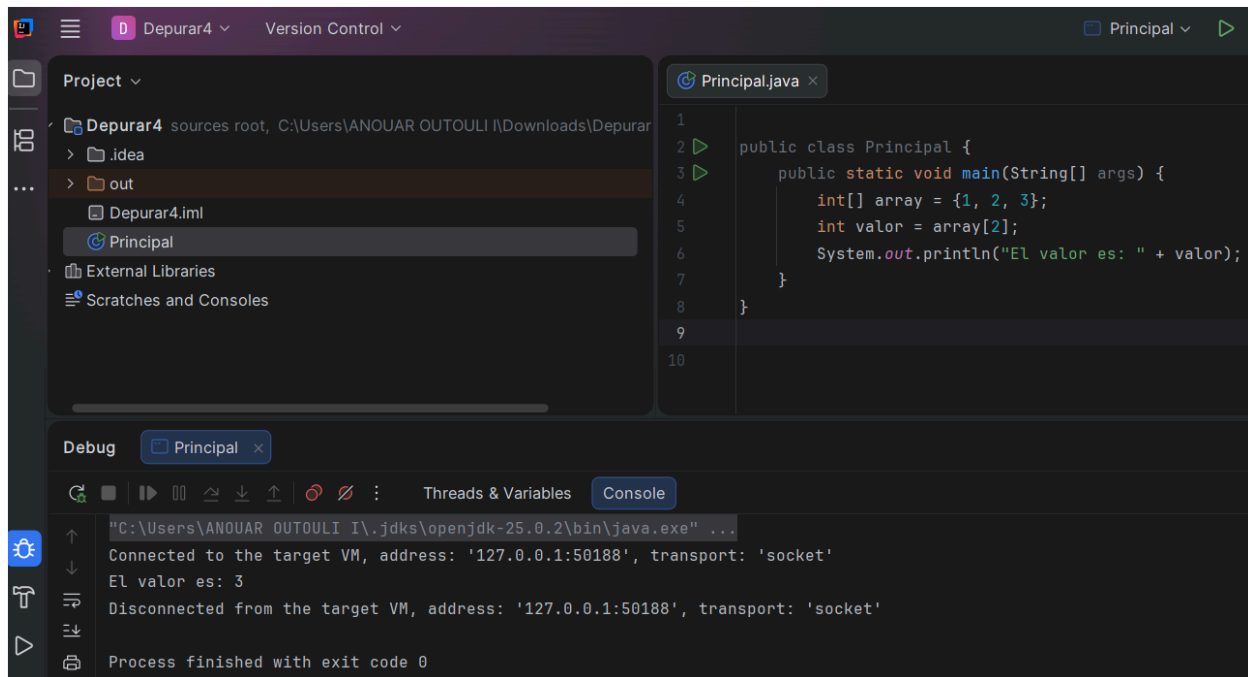
Los arrays en Java comienzan en el índice 0 y su último índice es `length - 1`.

Al utilizar un índice mayor que el tamaño del array, el programa genera un error en tiempo de ejecución.



El error se produce porque se intenta acceder a una posición del array que no existe.

El array tiene un tamaño de 3 elementos (índices de 0 a 2), pero se intenta acceder al índice 3, lo que provoca una excepción **ArrayIndexOutOfBoundsException**.



Actividad 9:

Al realizar el Build Project (CTRL+F9), IntelliJ IDEA compila el código fuente y genera el archivo .class correspondiente.

El bytecode se ha generado correctamente y se almacena en el directorio: out/production/Depurar4.

El archivo Principal.class puede abrirse desde el IDE, donde se muestra el código decompilado generado a partir del bytecode.

